Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Quoridor – Java

Rapport final

Abdeldjabar Ismail Abdraman, Cengiz Vasseur, Joris Douillet, Khadidja Ahmat Hassan, Matthieu Pourageaud

9 avril 2026

1 Explication détaillée du sujet

1.1 Historique du jeu Quoridor

Le Quoridor est un jeu de stratégie qui ne fait pas intervenir de hasard et où tous les joueurs connaissent parfaitement l'état du jeu à chaque instant. Il a été créé par Mirko Marchesi dans les années 1970. On peut le comparer à des classiques comme les échecs ou le go, où la réflexion et la stratégie priment complètement sur la chance. C'est pour cette raison qu'il est particulièrement intéressant pour l'étude de l'intelligence artificielle et de la théorie des jeux.

L'origine de Quoridor remonte à un jeu plus ancien appelé *Blockade*, créé par Philip Slater. Ce jeu introduisait déjà l'idée principale : utiliser des obstacles pour ralentir l'adversaire sur un plateau quadrillé. Mirko Marchesi a repris ce concept, l'a simplifié et équilibré pour qu'aucun joueur ne puisse être totalement bloqué. Dans Quoridor, un chemin vers la ligne d'arrivée doit toujours rester ouvert pour chaque joueur, ce qui distingue ce jeu de beaucoup d'autres jeux de blocage.

Le jeu a été édité et popularisé dans les années 1990 par la société Gigamic, spécialisée dans les jeux abstraits modernes. Cette période a vu l'apparition de nombreux jeux simples à comprendre mais très riches stratégiquement, et Quoridor s'inscrit parfaitement dans cette tendance avec son système minimaliste mais aux possibilités infinies.

En 1997, Quoridor a remporté le prix *Mind Game*, décerné par Mensa aux États-Unis, qui récompense les jeux demandant réflexion, logique et stratégie. Cette distinction a beaucoup contribué à sa notoriété internationale. Le jeu a ensuite été élu jeu de l'année dans plusieurs pays comme la France, les États-Unis, le Canada et la Belgique, confirmant son succès auprès des joueurs occasionnels comme des amateurs de stratégie.

Le principe du jeu est simple mais efficace : chaque joueur a un pion et doit atteindre la ligne opposée du plateau tout en posant éventuellement des murs pour ralentir ses adversaires. Cette dualité entre avancer soi-même et gêner les autres oblige à adopter des stratégies à la fois offensives et défensives. Contrairement à d'autres jeux abstraits où l'objectif est de capturer des pièces adverses, Quoridor repose uniquement sur la gestion de l'espace et du temps.

Le jeu est également intéressant d'un point de vue scientifique et informatique. Le plateau peut être modélisé comme un graphe : les cases sont des nœuds et les déplacements possibles des arêtes. Les murs viennent modifier ce graphe en supprimant certaines connexions, ce qui complique l'analyse du jeu. Cette représentation permet d'utiliser des algorithmes de parcours de graphe, comme le Breadth-First Search (BFS), pour calculer les chemins les plus courts vers l'objectif.

Quoridor est souvent utilisé pour tester différentes approches d'intelligence artificielle. Sa complexité combinatoire, moins élevée que celle des échecs ou du go, reste suffisante pour rendre inefficaces les méthodes de recherche exhaustive. Cela en fait un terrain idéal pour des algorithmes comme Minimax, l'élagage alpha-bêta ou le Monte Carlo Tree Search (MCTS). La règle obligeant à laisser un chemin libre ajoute une difficulté supplémentaire pour générer et valider les coups.

Plusieurs ouvrages analysent le jeu et ses stratégies. Glendenning[1] détaille les stratégies de base et insiste sur l'importance de bien choisir quand poser un mur et anticiper les actions de l'adversaire. Lloyd[2] s'est intéressé plus récemment à l'intelligence artificielle, en étudiant comment des algorithmes peuvent analyser et améliorer le jeu.

Avec l'avènement du numérique, Quoridor existe aussi sous forme d'applications mobiles, de plateformes en ligne et de projets académiques. Ces versions ont permis d'élargir le nombre de joueurs et de tester de nouvelles approches algorithmiques. Elles permettent aussi de collecter des données de jeu pour faire des analyses statistiques ou de l'apprentissage automatique.

Le jeu est aussi utilisé dans l'enseignement pour illustrer des concepts en informatique : graphes, algorithmes de recherche, complexité algorithmique ou systèmes multi-agents. Comme le souligne Bezakova[3], les jeux de plateau sont un excellent support pour apprendre la stratégie algorithmique et la logique.

Enfin, Quoridor est un excellent choix pour un projet informatique, car il combine plusieurs aspects : modélisation d'un système complexe, implémentation d'algorithmes de recherche, gestion de l'interface utilisateur et conception logicielle. Il est simple à comprendre mais offre une grande profondeur stratégique et un intérêt scientifique certain. Son histoire, sa popularité et les recherches qui lui sont consacrées montrent qu'il reste un jeu adapté aux défis contemporains, notamment en intelligence artificielle.

1.2 Principe général du jeu

Quoridor se joue sur un plateau carré composé de 9 × 9 cases dans sa version classique. Dans le cadre de notre projet, nous avons également envisagé des variantes avec des tailles différentes, allant de 3 × 3 à 15 × 15, à condition que la taille reste impaire. Cela permet de garder un bon équilibre entre les joueurs, car chacun part avec les mêmes chances.

Le jeu peut se jouer à deux, trois ou quatre joueurs. Dans une partie à 2, 3 ou 4 joueurs, chacun commence sur une ligne opposée du plateau. Chaque joueur possède un pion qui représente sa position sur le plateau, ainsi qu'un certain nombre de murs. Au total, il y a 20 murs dans la partie, répartis entre les joueurs.

Le but du jeu est simple : être le premier à atteindre la ligne opposée à celle de départ. Même si cet objectif paraît facile à comprendre, le jeu devient rapidement complexe, car les joueurs peuvent se gêner entre eux.

À chaque tour, un joueur a deux choix possibles. Il peut soit déplacer son pion, soit poser un mur. Le déplacement se fait d'une seule case à la fois, vers le haut, le bas, la gauche ou la droite, à condition qu'il n'y ait pas de mur qui bloque le passage. Ce déplacement permet d'avancer vers l'objectif.

L'autre possibilité est de poser un mur. Les murs servent à ralentir les adversaires en bloquant certains chemins. Ils doivent être placés entre les cases, soit horizontalement soit verticalement. Il est impossible de les placer en diagonale. Une fois posé, un mur ne peut plus être déplacé ni retiré. Cela oblige les joueurs à bien réfléchir avant de les utiliser, car ils sont en nombre limité.

Une règle très importante du jeu est qu'il est interdit de bloquer complètement un joueur. Autrement dit, il doit toujours exister au moins un chemin qui lui permet d'atteindre sa ligne d'arrivée. Cette règle est essentielle, car elle garantit que la partie peut toujours continuer jusqu'à ce qu'un joueur gagne.

Le jeu repose donc sur un équilibre entre avancer et bloquer les autres. Si un joueur se concentre uniquement sur sa progression, il risque de laisser ses adversaires avancer trop facilement. À l'inverse, s'il utilise trop de murs pour bloquer les autres, il peut ralentir sa propre progression ou se retrouver sans ressources en fin de partie.

Dans les parties à plusieurs joueurs, la stratégie devient encore plus intéressante. En effet, chaque joueur doit faire attention à plusieurs adversaires en même temps. Parfois, bloquer un joueur peut en aider un autre sans le vouloir. Il faut donc bien observer la situation et adapter sa stratégie en permanence.

D'un point de vue informatique, le plateau peut être vu comme un réseau de cases reliées entre elles. Les déplacements correspondent aux chemins possibles, et les murs viennent modifier ces chemins. Cela permet d'utiliser des algorithmes pour calculer les meilleurs trajets ou vérifier que les règles sont respectées.

En résumé, Quoridor est un jeu simple à comprendre mais difficile à maîtriser. Il demande à la fois de réfléchir à ses propres déplacements et d'anticiper ceux des autres joueurs. C'est ce mélange qui rend le jeu intéressant et qui explique pourquoi il est souvent utilisé dans des projets liés à l'intelligence artificielle.

1.3 Règles du jeu

Le Quoridor est un jeu de stratégie dont les règles sont simples à comprendre, mais dont la profondeur stratégique est très importante. Il se joue sur un plateau carré de 9 × 9 cases (des variantes existent avec des plateaux plus petits ou plus grands tant que la taille reste impaire comme nous l'avons fait ci-dessus). Le but principal est de déplacer son pion depuis sa ligne de départ jusqu'à la ligne opposée avant les autres joueurs. Le jeu se joue à deux ou à quatre joueurs, et chaque joueur dispose d'un nombre limité de murs pour ralentir ses adversaires. Pour gagner, il faut combiner habilement déplacements et placement de murs.

1.3.1 Déplacement des pions

Chaque joueur possède un pion qu'il déplace à son tour. Les déplacements sont très stricts : le pion peut avancer d'une seule case à la fois, soit vers l'avant, l'arrière, à gauche ou à droite. Il n'est jamais permis de déplacer son pion en diagonale sauf dans des situations spécifiques (que nous détaillerons plus bas). De plus, un pion ne peut jamais sortir du plateau.

Le pion ne peut pas passer à travers un mur. Si un mur se trouve directement sur le chemin, le pion est obligé de s'arrêter avant celui-ci. Cela signifie qu'un mur agit comme un véritable obstacle physique qui bloque le passage, ce qui rend le placement stratégique des murs crucial pour ralentir un adversaire.

Enfin, une règle fondamentale est qu'une case ne peut jamais contenir deux pions en même temps. Cela signifie qu'il n'est pas possible de "marcher sur un autre pion" ou de l'éliminer. Chaque pion occupe sa propre case et les joueurs doivent planifier leurs déplacements en tenant compte de la position des adversaires.

Cas particulier : saut par-dessus un pion Si le pion d'un autre joueur se trouve juste devant le pion du joueur actif, il est possible de le sauter. Ce saut permet de se déplacer de deux cases d'un coup, en "franchissant" le pion qui bloque le chemin. Cela ne peut se faire que si aucune autre barrière (mur ou bord du plateau) ne bloque directement l'espace derrière le pion à sauter.

Saut diagonal Dans le cas où le pion est bloqué par un autre pion et qu'un mur ou le bord du plateau empêche de sauter directement derrière le pion, le joueur peut effectuer un déplacement en diagonale. Cette règle donne une flexibilité stratégique intéressante, car elle permet d'éviter les situations où un pion serait complètement coincé par les autres. Les déplacements diagonaux ne sont donc pas une option de base mais une solution spéciale pour résoudre des situations de blocage (Fig. 1).

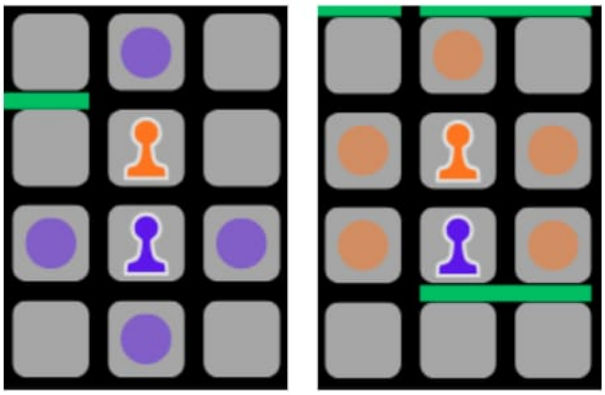


FIGURE 1 – Déplacement diagonal (saut d'un joueur)

1.3.2 Pose des murs

Chaque joueur dispose d'un certain nombre de murs à poser. Le placement des murs est soumis à des règles strictes qui garantissent l'équité du jeu et empêchent de bloquer complètement un adversaire.

Règles de base pour placer un mur Un mur doit toujours bloquer exactement deux cases adjacentes (voir Fig. 2 et Fig. 3). Il ne peut jamais être posé en diagonale ni traverser le plateau de façon irrégulière. Un mur posé reste définitivement sur le plateau et ne peut pas être déplacé ou retiré.

Un point fondamental est que le placement d'un mur ne doit jamais empêcher un joueur d'atteindre sa ligne d'arrivée. Autrement dit, il est impossible d'enfermer un joueur entre quatre murs ou de bloquer complètement toutes les routes possibles vers la victoire. Cette règle est cruciale pour que le jeu reste équitable et stratégique.

Impact stratégique des murs Les murs permettent de ralentir un adversaire ou de forcer un joueur à modifier sa trajectoire. Bien placés, ils peuvent obliger l'adversaire à prendre un chemin beaucoup plus long et ainsi offrir un avantage stratégique. Cependant, poser un mur n'est jamais gratuit : chaque joueur dispose d'un nombre limité de murs, il faut donc les utiliser avec parcimonie et intelligence.

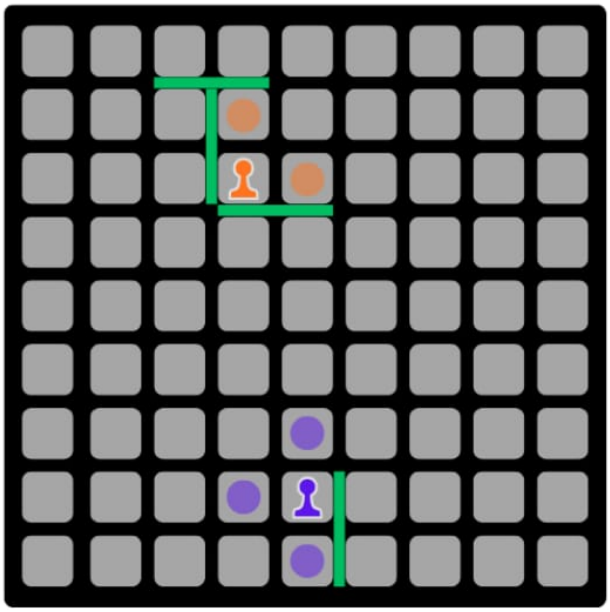


FIGURE 2 – Position de jeu normale

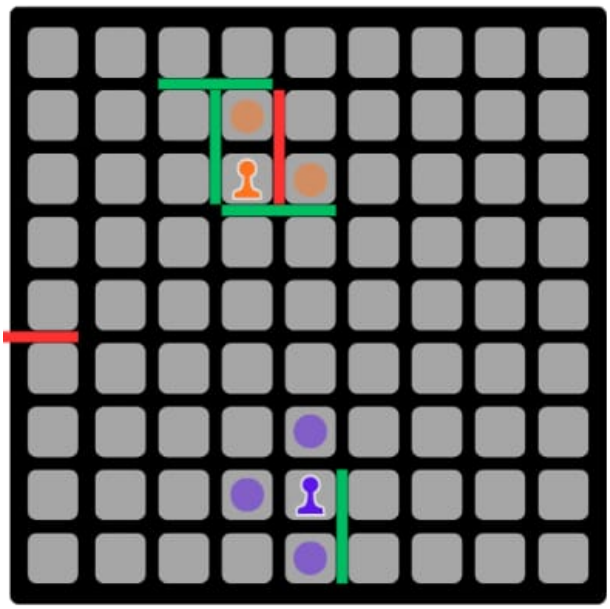


FIGURE 3 – Murs interdits

1.3.3 Interactions entre pions et murs

L'une des subtilités de Quoridor réside dans la combinaison des déplacements des pions et des murs. Le joueur doit toujours prévoir plusieurs coups à l'avance pour éviter de se retrouver bloqué et pour optimiser la trajectoire de son pion tout en gênant ses adversaires.

Par exemple, si un adversaire est proche de votre ligne de départ, il peut être judicieux de poser un mur pour l'obliger à faire un détour. À l'inverse, il peut être plus avantageux de concentrer ses murs sur des zones où plusieurs pions se croisent afin de créer des effets combinés, obligeant plusieurs adversaires à changer leur chemin.

Exemple concret Supposons qu'un joueur A a son pion devant un mur et que le joueur B est juste derrière lui. Le joueur B ne peut pas sauter par-dessus le mur pour atteindre directement l'autre côté, mais il pourra utiliser le saut diagonal si nécessaire. Les combinaisons de ces situations créent une profondeur stratégique très importante malgré la simplicité des règles de base.

1.3.4 Limitations et contraintes

Le Quoridor impose plusieurs contraintes pour maintenir l'équilibre du jeu et éviter des abus tactiques.

1. **Une seule case par pion** : chaque case ne peut être occupée que par un pion, empêchant les collisions. 2. **Murs fixes et non déplaçables** : un mur posé ne peut plus être retiré, ce qui oblige à réfléchir avant de le placer. 3. **Chemin obligatoire** : il doit toujours rester un chemin vers la ligne d'arrivée pour chaque joueur, garantissant que personne ne soit bloqué complètement. 4. **Déplacements limités** : un pion avance d'une seule case, sauf pour le saut par-dessus un adversaire ou le saut diagonal. 5. **Respect des bordures** : ni les pions ni les murs ne peuvent sortir du plateau.

1.3.5 Aspects tactiques

Ces règles simples cachent une richesse stratégique immense. Le joueur doit constamment prendre des décisions tactiques : avancer son pion, poser un mur pour gêner un adversaire, anticiper ses déplacements, utiliser les sauts et diagonales de manière optimale. La combinaison des déplacements simples et des murs crée un espace de décision très vaste, où chaque coup peut avoir des conséquences sur plusieurs tours à venir.

De plus, la limitation du nombre de murs et le fait que chaque joueur possède un seul pion rendent le jeu très équilibré. Le joueur doit toujours évaluer le rapport bénéfice/risque de poser un mur ou d'avancer son pion, car chaque décision influence directement la probabilité de victoire.

Exemple de stratégie Un joueur pourrait décider de ne pas poser de mur immédiatement pour avancer rapidement, mais cela peut permettre à un adversaire de progresser librement. À l'inverse, poser un mur tôt peut ralentir un adversaire, mais peut aussi le pousser à trouver un chemin plus court que prévu. La gestion de ces choix crée une tension permanente et rend chaque partie unique.

1.3.6 Conclusion sur les règles

En résumé, Quoridor repose sur des règles simples mais exigeantes : chaque pion se déplace d'une case, les murs sont limités et doivent être placés avec prudence, et il faut toujours laisser un chemin libre pour tous les joueurs. Les règles sur le saut et le saut diagonal ajoutent des subtilités qui enrichissent la stratégie. Cette combinaison de simplicité et de profondeur stratégique fait de Quoridor un jeu accessible mais riche, parfaitement adapté à l'étude des stratégies, à l'intelligence artificielle et à la réflexion logique.

1.4 Contraintes et difficultés du jeu

Le Quoridor est un jeu de stratégie dont la simplicité apparente cache des contraintes et des difficultés très importantes. Pour bien comprendre pourquoi ce jeu est si exigeant, il est nécessaire de détailler les différents types de limitations que chaque joueur doit gérer tout au long de la partie, ainsi que les défis stratégiques qu'elles impliquent. Ces contraintes touchent aussi bien la gestion des murs que les déplacements des pions, et elles s'intensifient lorsque le nombre de joueurs augmente.

1.4.1 Contraintes liées aux murs

L'une des contraintes majeures du Quoridor réside dans le nombre limité de murs que chaque joueur possède. Chaque mur représente une ressource rare et précieuse, et il n'est pas possible d'en poser autant que l'on souhaite. Cette limitation force le joueur à réfléchir à chaque placement et à évaluer soigneusement le moment où il sera le plus efficace. Poser un mur trop tôt peut ne pas avoir beaucoup d'impact, tandis que le poser trop tard peut permettre à un adversaire de prendre une avance irrattrapable.

De plus, la pose de murs est soumise à plusieurs règles strictes. Il est interdit de placer un mur qui bloque complètement l'accès à la ligne d'arrivée d'un joueur, ce qui oblige à calculer chaque coup pour s'assurer que personne n'est complètement enfermé. Cela signifie que le joueur ne peut pas simplement poser un mur "au hasard" pour ralentir un adversaire : il doit anticiper les conséquences de son action sur l'ensemble du plateau et sur tous les joueurs.

Chaque mur posé est permanent, ce qui renforce la pression sur le joueur. Une erreur de placement est irréversible et peut compromettre sa propre progression ou, dans certains cas, offrir un avantage stratégique à un adversaire. Cette permanence oblige les joueurs à combiner rigueur et anticipation dans leur planification.

1.4.2 Contraintes liées au mouvement des pions

Les déplacements des pions sont également soumis à des contraintes précises. Chaque pion ne peut se déplacer que d'une case à la fois, sauf dans le cas des sauts ou des déplacements diagonaux. Cette limitation impose au joueur de planifier ses trajets plusieurs tours à l'avance, car un mauvais choix peut le ralentir ou le forcer à contourner des obstacles, ce qui augmente le nombre de coups nécessaires pour atteindre la ligne d'arrivée.

L'interaction entre pions et murs crée une autre contrainte importante : un joueur doit toujours tenir compte de la position des autres pions. Si un adversaire se trouve directement devant lui, le joueur peut sauter par-dessus, mais uniquement si la case derrière l'adversaire n'est pas bloquée par un mur ou le bord du plateau. Dans le cas contraire, le joueur doit effectuer un saut diagonal. Ces règles introduisent des situations complexes où chaque déplacement nécessite une analyse des possibilités immédiates et futures.

1.4.3 Contraintes dans les parties à plusieurs joueurs

Le nombre de joueurs influence fortement la complexité et les contraintes du jeu. Dans une partie à deux joueurs, le joueur doit gérer uniquement un adversaire et peut donc anticiper plus facilement les mouvements et le placement de murs. En revanche, dans une partie à quatre joueurs, chaque joueur doit surveiller les déplacements et les stratégies de trois adversaires simultanément.

Cette augmentation du nombre de participants crée plusieurs difficultés supplémentaires. Tout d'abord, le joueur dispose de moins de contrôle sur l'évolution du plateau, car les autres participants peuvent poser des murs de manière imprévisible et modifier le trajet optimal. Ensuite, il devient beaucoup plus difficile de prévoir les conséquences de chaque coup, car un mur placé pour ralentir un joueur peut en même temps bénéficier à un autre adversaire en modifiant le chemin le plus court. Enfin, la gestion des murs devient plus stratégique : chaque mur doit être utilisé non seulement pour bloquer un adversaire mais aussi pour protéger sa propre progression.

1.4.4 Contraintes stratégiques et anticipatives

La vraie difficulté de Quoridor réside dans la nécessité d'adopter une stratégie à la fois anticipative et flexible. Un joueur ne peut pas se contenter de déplacer son pion de manière linéaire vers la ligne d'arrivée ou de poser des murs au hasard. Il doit constamment évaluer le plateau, prévoir les coups des adversaires et ajuster sa stratégie en fonction de la situation.

Cette planification implique de jongler entre deux objectifs parfois contradictoires : avancer son pion pour se rapprocher de la victoire et ralentir ses adversaires pour gagner du temps. Chaque décision a un coût et un bénéfice, et il faut savoir les équilibrer. Par exemple, poser un mur pour bloquer un adversaire peut rallonger le chemin du

joueur actuel si ce mur est mal placé, tandis qu'avancer trop rapidement son pion peut laisser l'adversaire libre de progresser sans contraintes.

Exemple concret Imaginons un joueur avec trois murs restants. Un adversaire avance rapidement et menace d'atteindre la ligne d'arrivée avant lui. Le joueur doit décider : poser immédiatement un mur pour le ralentir, ce qui risque de l'empêcher lui-même de suivre sa trajectoire optimale, ou économiser ses murs pour une situation ultérieure, en espérant pouvoir rattraper son retard autrement. Ce type de choix illustre la complexité stratégique du jeu.

1.4.5 Contraintes combinatoires et complexité

Le Quoridor est également complexe du point de vue combinatoire. Chaque tour offre plusieurs options : déplacer le pion dans une direction, poser un mur horizontal ou vertical sur une multitude de cases disponibles, ou sauter un adversaire. Cette grande variété de coups possibles fait que le nombre de positions envisageables augmente très rapidement, même après quelques tours seulement.

Cette explosion combinatoire rend le calcul exhaustif des coups très difficile, et c'est pourquoi des stratégies optimales ne peuvent être trouvées que par des méthodes avancées comme l'intelligence artificielle ou la planification algorithmique. Les joueurs humains doivent donc se baser sur des heuristiques et des prévisions partielles, ce qui ajoute une couche supplémentaire de difficulté et de créativité dans le jeu.

1.4.6 Contraintes psychologiques et décisionnelles

Au-delà des contraintes techniques et stratégiques, Quoridor impose des contraintes psychologiques importantes. Le joueur doit gérer la pression de prendre des décisions optimales à chaque tour, souvent sous la menace d'avancer plus lentement qu'un adversaire. Chaque mur posé ou chaque déplacement doit être réfléchi, car une erreur peut être coûteuse.

De plus, le joueur doit faire preuve de patience et d'adaptabilité. Une stratégie qui semblait efficace à un moment donné peut devenir inefficace à cause des actions des adversaires, et il faut savoir modifier son plan sans perdre de vue l'objectif final.

1.4.7 Résumé des contraintes

En résumé, les contraintes de Quoridor peuvent être classées en plusieurs catégories :

— **Contraintes de ressources :** nombre limité de murs, permanence des murs posés.
— **Contraintes de mouvement :** déplacement limité à une case, respect des règles de saut et diagonales.
— **Contraintes stratégiques :** équilibre entre avancer son pion et ralentir les adversaires.
— **Contraintes combinatoires :** grand nombre de coups possibles, explosion des positions envisageables.
— **Contraintes psychologiques :** nécessité d'anticipation, gestion de la pression et adaptation continue.

La combinaison de toutes ces contraintes rend le Quoridor extrêmement stimulant et difficile, tout en offrant une profondeur stratégique comparable à celle des échecs. La réussite dans ce jeu dépend de la capacité du joueur à gérer simultanément les ressources, les déplacements, l'anticipation des adversaires et l'évolution dynamique du plateau. C'est cette richesse et cette complexité qui en font un excellent support d'étude pour l'intelligence artificielle, la planification algorithmique et le développement de stratégies optimales.

1.5 Jeux ou implémentations existantes

Le jeu Quoridor, bien qu'il soit souvent associé à une version physique vendue en boutique, existe aujourd'hui sous de très nombreuses formes, que ce soit sous forme matérielle traditionnelle, dans des versions numériques, ou encore comme point de départ pour des projets informatiques ou de recherche. Cette diversité d'implémentations montre à la fois l'accessibilité du jeu et l'intérêt qu'il suscite, non seulement parmi les amateurs de jeux de société, mais aussi au sein de la communauté scientifique et de celle des développeurs.

Le jeu physique traditionnel Dans sa forme la plus classique, Quoridor est distribué par des éditeurs de jeux de société et se trouve facilement à l'achat dans de nombreuses boutiques spécialisées ou en ligne. Ce jeu se compose d'un plateau de 9×9 cases, de pions pour les joueurs et d'un ensemble de murs en bois ou en plastique. La simplicité de son matériel contraste avec la profondeur stratégique qu'il offre. C'est un jeu abstrait accessible qui peut être appris en quelques minutes, mais qui peut demander des dizaines d'heures d'entraînement pour être maîtrisé. Cette accessibilité et cette profondeur expliquent en grande partie pourquoi il a su conquérir un public large, des joueurs occasionnels aux passionnés de stratégie.

L'un des signes de la reconnaissance du jeu est son prix *Mind Game*, décerné en 1997 par l'association Mensa, qui récompense les jeux faisant appel à la réflexion et à la logique. Cette distinction a contribué à sa diffusion internationale, ainsi qu'à sa réputation comme un jeu stimulant et intellectuellement exigeant.

Versions en ligne et communautés de joueurs Avec le développement des technologies numériques, Quoridor a très vite été adapté en versions informatiques. Aujourd'hui, il existe de nombreuses plateformes en ligne où il est possible de jouer à Quoridor gratuitement contre d'autres joueurs à travers le monde. Ces environnements en ligne offrent plusieurs avantages par rapport à la version physique :

— **Accessibilité immédiate** : nul besoin d'acheter un plateau physique ou de trouver un adversaire en face de soi ; il suffit d'un navigateur ou d'une application.
— **Matchmaking automatisé** : beaucoup de versions en ligne proposent de jumeler les joueurs par niveau, ce qui permet d'affronter des adversaires de force comparable et d'améliorer progressivement son jeu.
— **Analyse des parties** : certaines plateformes offrent un historique des parties ou un système de replay permettant de revoir les coups joués, ce qui est extrêmement utile pour apprendre et s'améliorer.
— **Statistiques et classement** : des systèmes de classement ELO ou similaires permettent de suivre sa progression et de se mesurer à une communauté plus large.

Ces adaptations numériques ont grandement contribué à la popularité actuelle de Quoridor, en particulier auprès d'un public jeune et connecté. Elles ont aussi facilité l'organisation de tournois, de défis et d'événements en ligne autour du jeu, ce qui a favorisé l'émergence d'une communauté active autour de Quoridor.

Implémentations open-source et projets personnels Au-delà des versions commerciales et des plateformes en ligne, le jeu a inspiré de nombreux passionnés à créer leurs propres implémentations. Sur des plateformes de partage de code telles que GitHub, il est possible de trouver plusieurs projets qui reproduisent ou étendent Quoridor dans des langages de programmation variés.

Deux exemples de projets particulièrement intéressants sont :
— **Le dépôt de *alainrinder*** : disponible à l'adresse https://github.com/alainrinder/quoridor.py, ce projet propose une implémentation en Python de Quoridor avec une interface graphique. Il permet de jouer contre une IA basique et constitue une base solide pour quiconque souhaite étudier le jeu ou développer une IA plus avancée.
— **Le dépôt de *pavlosdais*** : disponible sur https://github.com/pavlosdais/Quoridor, ce projet propose une version du jeu en langage C fonctionnant dans le terminal. Ce type de projet est particulièrement utile pour comprendre les mécanismes internes du jeu et implémenter des moteurs de décision légers.

Ces projets sont des ressources précieuses pour les développeurs, car ils permettent de manipuler directement les structures de données associées aux plateaux, aux pions et aux murs. Ils servent aussi de point de départ pour expérimenter différentes stratégies d'IA ou pour intégrer le jeu dans des environnements plus larges.

En complément de ces projets open-source, on trouve également des tutoriels, des vidéos explicatives et des projets didactiques réalisés par des amateurs. Par exemple, une vidéo YouTube montre comment coder le jeu en Python en utilisant un environnement Jupyter Notebook (voir la vidéo). Ce type de contenu est particulièrement utile pour les débutants qui souhaitent comprendre les bases de l'implémentation du jeu sans partir de zéro.

Travaux scientifiques sur Quoridor et agents de jeu Quoridor n'est pas seulement un jeu de société apprécié des joueurs ; il a aussi attiré l'attention des chercheurs en intelligence artificielle et en informatique théorique. La raison principale est que, malgré sa simplicité apparente, Quoridor offre une complexité stratégique significative qui en fait un excellent test pour des méthodes d'IA.

L'un des premiers travaux scientifiques sur ce sujet est celui de Mertens, qui s'intéresse à la conception d'un agent capable de jouer à Quoridor[4]. Ce travail explore les défis associés à la modélisation du jeu, à la génération de coups légaux et à la mise en place d'une stratégie compétitive.

Par la suite, plusieurs études ont adopté des méthodes plus sophistiquées pour aborder ce problème. L'une des approches dominantes dans la recherche sur les jeux combinatoires est le *Monte Carlo Tree Search* (MCTS), un algorithme qui a démontré des performances remarquables sur des jeux complexes comme le go, et qui a été appliqué avec succès à Quoridor[5, 6]. Cet algorithme utilise des simulations statistiques pour estimer la valeur des positions de jeu et choisir les coups les plus prometteurs, ce qui en fait un candidat naturel pour des jeux à haute complexité stratégique comme Quoridor.

Un autre champ important de la recherche est l'étude des heuristiques et des fonctions d'évaluation spécifiques à ce jeu. Par exemple, des travaux ont tenté de définir des mesures qui estiment la distance effective d'un pion à sa ligne d'arrivée en tenant compte des murs déjà placés, ou encore de combiner plusieurs métriques pour guider la recherche de coups dans des espaces de décision très vastes.

En plus des méthodes de recherche locale comme MCTS, d'autres techniques issues de l'intelligence artificielle ont été explorées, comme :
— **Les algorithmes Minimax et alpha-bêta**, classiques dans les jeux à deux joueurs, qui explorent l'arbre des coups possibles pour déterminer les actions optimales en partant de la logique que l'adversaire joue parfaitement.
— **Les méthodes hybrides**, qui combinent des heuristiques locales, des simulations et des approximations de valeur pour accélérer la prise de décision.
— **L'apprentissage automatique**, où des modèles peuvent être entraînés à partir de données de parties pour apprendre des stratégies gagnantes sans programmation explicite des règles tactiques.

Le livre de Lloyd sur Quoridor explore ces aspects sous un angle plus moderne, en discutant de l'intégration d'algorithmes d'intelligence artificielle dans l'analyse stratégique du jeu[2]. Il constitue une ressource intéressante pour comprendre comment les concepts théoriques d'IA se traduisent dans la pratique du jeu.

Ressources pédagogiques et didactiques Au-delà des travaux scientifiques et des implémentations logicielles, Quoridor est utilisé dans des contextes pédagogiques pour enseigner des concepts d'informatique. Par exemple, comme le souligne Bezakova, les jeux de plateau servent souvent d'introduction aux stratégies algorithmiques dans les cours d'informatique, car ils permettent d'illustrer des idées comme la recherche de plus court chemin, l'exploration d'arbres de décision ou la conception de moteurs de jeu[3].

Dans ce cadre, les étudiants peuvent s'initier à la notion de parcours de graphe pour calculer les chemins optimaux autour des murs, ou encore à la notion d'heuristique pour guider la prise de décision dans un espace de jeu très large. Ces exercices pratiques permettent de relier des concepts abstraits d'algorithmique à des situations concrètes, ce qui facilite l'apprentissage et la compréhension.

Comparaison avec d'autres jeux et plateformes d'étude Quoridor se distingue aussi lorsqu'on le compare à d'autres jeux utilisés dans les recherches en IA. Par exemple, des jeux comme les échecs ou le go ont des espaces de décision extrêmement vastes, mais ils reposent sur des mécaniques de capture et de confrontation directes. Quoridor, en revanche, repose entièrement sur la gestion de l'espace et des ressources (les murs), ce qui en fait un bon terrain d'étude pour des problèmes comme la planification sous contraintes, l'optimisation de trajectoires, ou encore la coopération et la compétition multi-agents.

Des plateformes de recherche multi-jeux ont même intégré Quoridor comme l'un des environnements de test pour comparer les performances d'algorithmes génériques. Cela permet de mesurer à quel point une technique d'IA est capable de s'adapter à des jeux avec des règles différentes et des structures stratégiques variées.

En résumé, Quoridor est bien plus qu'un simple jeu de société. Il existe aujourd'hui sous de multiples formes — physique, numérique, open-source et scientifique — chacune adaptée à un usage spécifique. Que ce soit pour jouer contre des amis, affronter des adversaires en ligne, étudier des stratégies d'intelligence artificielle ou enseigner des concepts informatiques, Quoridor propose un terrain riche et stimulant.

Les implémentations disponibles, qu'elles soient grand public ou issues de projets personnels, montrent l'intérêt pratique du jeu, tandis que les travaux scientifiques démontrent sa pertinence comme objet d'étude. Enfin, l'utilisation de Quoridor dans des contextes pédagogiques renforce son rôle comme outil d'apprentissage de l'algorithmique et de l'intelligence artificielle. Grâce à cette diversité, Quoridor reste un jeu vivant, apprécié et étudié dans de nombreux domaines.

1.6 Objectifs du projet réalisés

Quelques objectifs qu'il était plus que nécessaire et une évidence de réaliser afin de proposer une expérience de jeu cohérente et satisfaisante étaient :

- Afficher le plateau selon ses configurations (taille, position des pions et murs, murs restants) ;
- Gérer les coups disponibles à chaque tour des joueurs (coups autorisés, coups illégaux, requêtes erronées, contraintes des murs) ;
- Adapter la partie selon les configurations demandées (interface clavier/graphique, mode blitz/en ligne, adversaire humain/IA et leur nombre, nombre de murs, ...) ;
- Proposer une langue adaptative selon la région du joueur (au moins anglais et français) ;

La réalisation de ces objectifs devait être notre première priorité sans quoi il aurait été compliqué de parvenir à la fin de ce projet. Nous avons alors pu accomplir ces tâches afin de poursuivre vers des objectifs plus poussés, dont nous dirions même plus challengeant.

1.7 Fonctionnalités attendues dans le cahier des charges.

Le projet requiert 15 généralités que nous avons respectés et qui auront été notre base de départ afin de fournir un code complet et rigoureux :

- Langage de programmation : JAVA SE 21+ ;
- Style de codage : Coding style de Google ;
- Documentation : Javadoc ;
- Langue par défaut du code : Anglais ;
- Système crible : Fonctionne sur des systèmes GNU/Linux ;
- Tests : Tester et valider le code (à hauteur de 85
- Bugs : Aucun bug ne fait crasher le programme et un message d'erreur explicite est affiché à l'utilisateur ;
- Performances : Mesurer et documenter sur différentes entrées ainsi que ses cas limites ;
- Bibliothèques : Seul la bibliothèque du langage et celle listées sont acceptées ;
- Build-system : Maven ;
- Framework de tests : JUnit + Jacoco ;
- Gestion des options : Bibliothèque commons-cli ;
- Bibliothèque graphique : JavaFx ;
- Internationalisation : ResourceBundle et Locale ;

— Apprentissage automatique : Bibliothèque TensorFlow (Java API) ;

Le projet nécessitait également la réalisation de multiples tâches réparties en 40 besoins dont nous avons précisé l'avancement de chacun des 40 en annexe à la fin de ce document comprenant les objectifs cités dans la section précédente 1.6 Objectifs du projet réalisés.

2 Les Fonctionnalités étendues

2.1 Fonctionnalité 1 : Réseau client/serveur et gestion multi-clients

2.1.1 Dimension réseau du projet Quoridor

L'un des objectifs principaux de notre projet consistait à étendre le jeu Quoridor à une dimension réseau, afin de permettre à plusieurs joueurs de s'affronter à distance. Cette fonctionnalité transforme radicalement la manière dont le jeu peut être pratiqué : il ne s'agit plus uniquement d'un jeu local sur un ordinateur unique, mais d'une application distribuée dans laquelle plusieurs utilisateurs interagissent via leurs machines respectives, tout en partageant un état de jeu centralisé. Cette extension réseau implique non seulement de gérer les aspects techniques liés aux communications entre machines, mais également de garantir que les règles du jeu soient respectées de manière cohérente pour tous les participants, indépendamment de leur localisation.

2.1.2 Première étape : serveur simple et tests de connectivité

La première étape de cette implémentation réseau a consisté à mettre en place un serveur simple. Ce serveur devait être capable d'accepter une seule connexion cliente à la fois et de traiter des commandes élémentaires. L'objectif initial n'était pas de gérer plusieurs parties ou joueurs simultanément, mais de créer un prototype fonctionnel permettant de tester la connectivité entre un client et un serveur, et de vérifier que les échanges de messages étaient correctement interprétés.

Pour cela, nous avons défini des commandes de base telles que `PING` et `QUIT`. La commande `PING` permet au client de tester si le serveur est actif et répond correctement aux requêtes, renvoyant un `PONG` pour confirmer la disponibilité du serveur. La commande `QUIT` permet au client de terminer la session proprement, ce qui entraîne la fermeture de la connexion côté serveur et libère les ressources associées. Cette étape de validation était essentielle pour s'assurer que les échanges réseau étaient fiables et que les fondations pour un développement multi-joueurs ultérieur étaient solides.

En parallèle, un diffuseur UDP simple a été implémenté pour permettre au serveur d'annoncer sa présence sur le réseau local. Cette annonce contient le nom du serveur et le port TCP sur lequel il est joignable. Côté client, une méthode dédiée écoute ces messages et construit une liste des serveurs disponibles. Bien que ce mécanisme reste basique et davantage adapté à des tests locaux qu'à une découverte réseau complète, il a permis de mettre en place une interaction automatisée entre clients et serveurs et d'éviter la configuration manuelle fastidieuse des adresses et ports.

2.1.3 Deuxième étape : multi-clients et multi-parties

Une fois le serveur simple opérationnel, le projet a évolué vers une architecture capable de gérer plusieurs clients et plusieurs parties simultanément. L'objectif principal était de permettre à différents joueurs de se connecter, de créer ou rejoindre des parties, et de jouer sans que leurs actions interfèrent avec d'autres parties actives.

Pour cela, la classe `MultiGameServer` a été conçue. Contrairement au serveur simple, cette version est capable de gérer plusieurs connexions simultanées grâce à l'usage de threads distincts ou de mécanismes de programmation asynchrone. Chaque client connecté dispose ainsi de son propre canal de communication avec le serveur. De plus, le serveur conserve l'état de plusieurs sessions de jeu, ce qui permet à plusieurs parties de se dérouler en parallèle, chacune avec son tableau de scores et sa liste de joueurs. Le serveur joue ici un rôle d'autorité centrale : il reçoit toutes les commandes des clients, valide chaque action et met à jour l'état officiel du jeu avant de notifier les autres joueurs concernés. Cette centralisation est essentielle pour Quoridor, où les règles sont strictes et chaque coup doit être vérifié pour éviter toute incohérence, comme la pose d'un mur qui bloquerait totalement un joueur.

2.1.4 Protocole de communication et format des messages

Le choix du protocole de communication s'est porté sur un modèle simple, basé sur l'échange de chaînes ASCII terminées par un retour à la ligne. Cette approche présente plusieurs avantages : les messages sont faciles à lire, à comprendre et à déboguer, ce qui est particulièrement utile pendant le développement et les tests. Les commandes principales incluent :

— `HELLO` : initialisation de la connexion entre le client et le serveur.
— `PING` : vérification de la disponibilité du serveur.
— `QUIT` : fermeture propre de la session client.
— `MOVE` : envoi d'un coup effectué par le joueur.

Les réponses du serveur incluent des messages tels que `WELCOME`, `PONG`, `BYE`, `OK`, `ERROR`, ainsi que des notifications spécifiques au jeu comme `GAME_START`, `OPPONENT_MOVE` et `GAME_OVER`. Ce format simple garantit une compréhension claire des échanges et permet au client d'interpréter rapidement les messages pour mettre à jour son interface graphique ou sa logique de jeu.

2.1.5 Découverte et annonce des serveurs

Pour rendre le jeu accessible sur un réseau local sans configuration manuelle, une classe `ServerBroadcaster` a été implémentée. Cette classe envoie régulièrement des messages UDP contenant le nom du serveur et son port TCP. Les clients, via la méthode `serverList()` de `GameClient`, écoutent ces messages et détectent les serveurs disponibles sur le réseau. Cela permet aux joueurs de choisir un serveur actif sans connaître préalablement son adresse IP. Bien que cette fonctionnalité soit encore limitée et davantage orientée vers des tests locaux, elle constitue une base solide pour des améliorations futures, notamment la découverte automatique sur de véritables réseaux étendus.

2.1.6 Validation des coups côté serveur

Un aspect central de l'architecture réseau du projet est que toutes les actions de jeu sont validées côté serveur. Lorsqu'un joueur envoie un coup, celui-ci est d'abord transmis sous forme de chaîne de caractères. La classe `NetworkMovePar` se charge de transformer cette chaîne en objet métier `Move` reconnu par le moteur du jeu. Le serveur utilise ensuite la logique métier contenue dans le package `core` pour vérifier la légalité du coup : déplacements valides, respect des contraintes des murs, autorisation de sauts ou de déplacements diagonaux, et respect des règles de progression vers la ligne d'arrivée.

Seule une fois validé, le coup est accepté et retransmis aux autres joueurs de la partie via le serveur. Cette centralisation garantit que tous les clients partagent une vision cohérente du plateau de jeu, évitant les incohérences qui pourraient survenir si chaque client appliquait ses propres règles localement.

2.1.7 Gestion des sessions et synchronisation

La gestion des sessions multi-parties implique également de maintenir une synchronisation stricte entre tous les participants. Chaque partie dispose d'un identifiant unique et d'une liste de joueurs connectés. Le serveur conserve un état complet de la partie, incluant la position des pions, la disposition des murs et le nombre de coups réalisés. Lorsqu'un joueur effectue une action, le serveur met à jour cet état et envoie une notification aux autres participants pour qu'ils puissent afficher la modification en temps réel sur leurs interfaces.

Cette approche permet également de gérer les cas d'exception, tels que la déconnexion d'un joueur. Le serveur peut alors suspendre la partie, attendre le retour du joueur, ou déclarer une victoire par abandon selon les règles définies. De plus, elle permet de maintenir l'intégrité des données en cas de conflits ou d'envois simultanés de coups par différents clients.

2.1.8 Évolutivité et architecture future

L'architecture client/serveur adoptée pour le projet offre une grande flexibilité pour des évolutions futures. Il est par exemple possible d'intégrer des fonctionnalités supplémentaires telles que :

— un système de matchmaking pour associer automatiquement les joueurs selon leur niveau,
— des parties classées avec calcul de scores et de statistiques,
— l'intégration d'IA côté serveur pour remplacer un joueur absent ou compléter une partie,
— un suivi historique des parties pour permettre aux utilisateurs de revoir leurs stratégies,
— une sécurité renforcée avec authentification et chiffrement des communications.

En adoptant une architecture modulaire, dans laquelle la communication réseau est séparée de la logique métier et des interfaces, il est possible de faire évoluer le projet sans perturber les composants existants. Chaque module (réseau, interface, IA, logique métier) peut être testé et amélioré indépendamment.

2.1.9 Résumé de l'approche

En résumé, l'implémentation réseau de Quoridor repose sur :

— une architecture client/serveur centralisée garantissant la cohérence des règles,
— un protocole simple basé sur des chaînes ASCII pour faciliter le développement et le débogage,
— la gestion de plusieurs clients et parties simultanées via `MultiGameServer`,
— une découverte basique de serveurs via UDP avec `ServerBroadcaster`,
— la validation stricte des coups côté serveur via le moteur de jeu et `NetworkMoveParser`,
— une synchronisation continue des états entre clients pour une expérience de jeu fluide et cohérente.

Cette approche permet non seulement de jouer à distance avec des amis ou d'autres utilisateurs, mais également de poser les bases d'une plateforme de jeu évolutive, modulable et sécurisée, tout en respectant strictement les règles du Quoridor original.

2.1.10 Structures de données utilisées

La partie réseau s'appuie sur plusieurs structures de données importantes.

Dans `MultiGameServer`, les clients sont stockés dans une structure `Map<String, ClientSession>`. Chaque client possède un identifiant unique (C1, C2, etc.).

Les parties actives sont stockées dans `Map<String, ServerGameSession>`.

Le tableau des scores repose sur `Map<String, ScoreEntry>`.

La synchronisation repose sur un verrou utilisé avec `synchronized(lock)`.

Côté client, `GameClient` utilise une `SynchronousQueue<String>` pour gérer les réponses synchrones.

2.1.11 Algorithme mis en place

Le fonctionnement du réseau peut être décrit en plusieurs étapes.

Lorsqu'un serveur simple ou multi-clients est démarré, il ouvre d'abord une socket TCP sur un port donné. Il lance ensuite un thread de diffusion UDP chargé d'annoncer périodiquement sa présence, puis un thread de surveillance chargé de détecter les clients inactifs. Dans le cas du serveur multi-clients, une boucle principale accepte ensuite chaque nouvelle connexion TCP, crée un objet ClientSession, lui attribue un identifiant unique et démarre un thread dédié à la gestion de ce client.

Lorsqu'un client envoie une commande, le serveur lit la ligne reçue et décide de l'action à exécuter. Une commande `HELLO` enregistre le nom du joueur. Une commande `PING` entraîne l'envoi de PONG. Une commande `QUIT` provoque l'envoi de `BYE` puis la fermeture de la session. Une commande `MOVE` déclenche le traitement principal.

Pour un coup de jeu, le serveur récupère d'abord la partie associée au client. Il vérifie ensuite que le joueur appartient bien à une partie active et que c'est bien son tour. Le texte du coup est alors converti en objet Move via NetworkMoveParser. Si le format est invalide, le serveur renvoie une erreur. Si le format est correct, le coup est transmis à la session de jeu côté serveur, qui s'appuie elle-même sur le moteur du jeu pour vérifier sa légalité. Si le moteur refuse le coup, le serveur renvoie `ERROR illegal move`. Sinon, il renvoie OK au joueur concerné, puis diffuse `OPPONENT_MOVE` etc. aux autres joueurs de la partie.

Après chaque coup valide, le serveur vérifie si la partie est terminée. Si un gagnant est détecté, il met à jour le tableau des scores, réinitialise le statut des joueurs, envoie `GAME_OVER WIN` au vainqueur et `GAME_OVER LOSE` aux autres joueurs, puis supprime la partie de la table des parties actives.

En parallèle, un thread de timeout examine régulièrement la liste des clients. Si un client reste inactif plus d'une minute, il est considéré comme expiré, reçoit un message d'erreur, puis est déconnecté. Si ce client participait à une partie, les autres joueurs sont remis à l'état disponible et avertis de la déconnexion.

Chaque connexion crée une `ClientSession`. Lorsqu'un client envoie une commande :

— `HELLO` : enregistre le joueur
— `PING` : réponse `PONG`
— `QUIT` : réponse `BYE`
— `MOVE` : traitement du coup

Pour un coup :

— Vérification du joueur et du tour
— Conversion en `Move`
— Validation par le moteur de jeu

Si le coup est valide :

— Envoi de `OK`
— Diffusion de `OPPONENT_MOVE`

Fin de partie :

— Mise à jour du score
— Envoi de `GAME_OVER WIN` ou `GAME_OVER LOSE`

2.1.12 Synchronisation entre serveur et client

Un point important de l'algorithme mis en place concerne la synchronisation entre la partie distante et sa représentation locale côté client. Le serveur conserve l'état officiel de la partie et reste seul responsable de la validation des actions. Le client, quant à lui, n'a pas pour rôle de décider si un coup est légal ou non dans le cadre du protocole réseau ; il envoie une demande d'action et attend la réponse du serveur. Si le coup est accepté, il est alors appliqué localement afin de refléter fidèlement la progression réelle de la partie. Si le serveur refuse le coup, le client conserve son état précédent et affiche simplement le message d'erreur reçu. Ce mécanisme évite les divergences entre l'état du client et celui du serveur.

Cette synchronisation concerne également les coups joués par les autres participants. Lorsqu'un joueur adverse effectue une action valide, le serveur transmet cette information aux autres clients concernés. Chaque client reçoit alors un message réseau, convertit le texte correspondant en objet métier, puis l'applique à son propre moteur local.

Ainsi, même si chaque client possède une représentation locale du plateau, cette représentation reste construite à partir des validations émises par le serveur, ce qui garantit la cohérence de la partie distribuée.

2.1.13 Intégration dans les interfaces

Depuis les premières versions de cette fonctionnalité, l'intégration réseau a été étendue dans les différentes interfaces du projet. En mode ligne de commande, les commandes réseau ne servent plus uniquement à établir une connexion, mais également à maintenir la partie locale cohérente avec les messages reçus depuis le serveur. Lorsqu'un coup est envoyé par le client et validé côté serveur, il est ensuite rejoué localement afin que l'état affiché corresponde bien à l'état officiel de la partie. De la même manière, lorsqu'un coup adverse est reçu, il est interprété puis appliqué localement pour conserver la synchronisation entre les deux côtés.

Dans l'interface graphique, cette intégration a également été mise en place. Le joueur peut rejoindre un serveur, définir son nom, lancer une partie réseau et jouer directement depuis le plateau graphique. Contrairement au mode local, les actions effectuées dans l'interface ne sont pas appliquées immédiatement par le client. Elles sont d'abord converties en messages réseau puis envoyées au serveur, qui conserve le rôle d'autorité centrale. Une fois la réponse reçue, l'interface met à jour le moteur de jeu local et rafraîchit l'affichage. Cette organisation permet de réutiliser le moteur existant tout en gardant une séparation claire entre l'interface utilisateur, la communication réseau et la validation des règles.

2.1.14 Lien avec les autres composants du projet

Cette fonctionnalité réseau ne repose pas sur une logique isolée, mais s'appuie directement sur plusieurs structures et algorithmes déjà présents dans le noyau du projet. Elle mobilise notamment la représentation des positions, des déplacements et des murs, la gestion de l'état de la partie, la validation des coups ainsi que les mécanismes de calcul d'accessibilité utilisés par le moteur de jeu.

Le réseau agit donc comme une couche supplémentaire de communication et de coordination, sans remplacer les règles métiers déjà implémentées dans le cœur du projet.

2.1.15 Difficultés rencontrées

La principale difficulté de cette fonctionnalité a été la gestion de la concurrence. En effet, dans la version multi-clients, le serveur fonctionne avec plusieurs threads : un thread principal d'acceptation des connexions, un thread par client, un thread pour le diffuseur UDP et un thread pour la surveillance des timeouts. Sans mécanisme de protection, plusieurs accès simultanés aux listes de clients, de parties ou de scores auraient pu provoquer des incohérences. L'utilisation d'un verrou commun avec synchronized (lock) a donc été nécessaire.

Une deuxième difficulté a concerné la gestion des réponses synchrones et des messages asynchrones côté client. Certaines commandes, comme PING ou QUIT, attendent une réponse précise et immédiate. En parallèle, d'autres messages peuvent arriver à tout moment depuis le serveur, par exemple WELCOME, OPPONENT_MOVE ou GAME_OVER. Il a donc fallu concevoir un client capable de lire les messages en continu tout en permettant à certaines méthodes d'attendre une réponse particulière. Cette difficulté a été traitée dans GameClient avec un thread d'écoute et une file de synchronisation.

Une troisième difficulté a été l'intégration du protocole réseau avec la logique métier du jeu. Le serveur ne se contente pas de relayer les messages ; il doit interpréter un coup reçu sous forme textuelle, le convertir correctement en objet métier et vérifier sa légalité avec le moteur du jeu. Cela impose une bonne articulation entre les classes réseau et les classes du noyau du projet.

Enfin, la découverte de serveurs a demandé une conception séparant deux types d'échanges : le TCP, utilisé pour la communication fiable entre client et serveur pendant la partie, et l'UDP, utilisé pour l'annonce de présence du serveur. Dans l'état actuel du projet, cette partie existe bien mais reste encore partiellement finalisée.

2.1.16 Difficulté d'intégration entre réseau et interfaces

Une difficulté supplémentaire a concerné l'intégration concrète du réseau dans les interfaces déjà existantes du projet. En effet, le moteur de jeu avait initialement été conçu pour fonctionner en mode local, où une action utilisateur peut être appliquée directement. Dans le mode réseau, ce fonctionnement doit être adapté : l'action saisie par le joueur doit d'abord être transformée en message texte, envoyée au serveur, puis éventuellement rejouée localement seulement après validation. Cela impose une logique différente de celle du mode local, car l'interface doit être capable d'attendre une confirmation distante avant de modifier son propre état.

Cette contrainte est particulièrement visible dans l'interface graphique, où l'utilisateur interagit directement avec le plateau. Il a donc fallu faire en sorte que les clics et déplacements effectués sur le plateau ne déclenchent pas immédiatement une modification définitive du jeu, mais passent d'abord par la couche réseau. Cette adaptation a demandé de bien articuler les responsabilités entre l'interface, le client réseau et le moteur de jeu, afin de garder une expérience cohérente tout en respectant le rôle central du serveur.

2.1.17 Tests réalisés

La fonctionnalité réseau a été accompagnée de plusieurs tests unitaires.

La classe `GameServerTest` vérifie le fonctionnement du serveur simple. Les tests confirment qu'un client peut se connecter, qu'un message PING reçoit bien PONG, qu'une commande QUIT reçoit BYE, et qu'une commande inconnue reçoit une réponse générique.

La classe `GameClientTest` vérifie les comportements essentiels du client. Les tests contrôlent la réussite ou l'échec d'une connexion, l'affichage d'un message d'erreur lorsqu'aucune connexion n'est active, l'envoi correct des commandes HELLO, MOVE et QUIT, ainsi que la commande serverList() à l'aide d'un faux serveur UDP.

La classe `MultiGameServerTest` couvre une partie importante de la logique multi-clients. On y retrouve notamment des tests sur l'état du serveur, la liste des joueurs, le tableau des scores, la création de parties à deux ou quatre joueurs, le changement de statut des joueurs, la gestion des tours, les erreurs liées à un joueur absent ou indisponible, ainsi que certains comportements liés à la validation des coups, à leur diffusion et à la fin d'une partie.

D'autres tests existent également autour de classes connexes comme `NetworkMoveParserTest, ServerBroadcaster` `ClientSessionTest, ServerGameSessionTest et NetworkLauncherTest`. Dans l'ensemble, ces tests montrent que la partie réseau n'a pas seulement été commencée, mais qu'elle a réellement été structurée et vérifiée.

2.1.18 Résultat final

Au stade actuel du projet, la fonctionnalité réseau est bien avancée et constitue une partie importante du système. Le projet contient un serveur simple, ainsi qu'un serveur multi-clients permettant de gérer plusieurs joueurs simultanément, de créer des parties à deux ou quatre joueurs, de maintenir un tableau des scores et de valider les coups à distance avant leur diffusion.

Le résultat obtenu montre que l'architecture du projet est suffisamment modulaire pour réutiliser le moteur de jeu existant dans un contexte réseau. Cette réutilisation est un point fort, car elle évite de dupliquer les règles du jeu entre le mode local et le mode distant.

Cependant, l'implémentation actuelle ne couvre pas encore parfaitement l'ensemble du cahier des charges. La gestion multi-clients et multi-parties est globalement réalisée et fonctionnelle. L'intégration réseau dans le mode ligne de commande est opérationnelle, et l'intégration dans l'interface graphique permet également de rejoindre un serveur, de lancer une partie réseau, d'envoyer des coups et de mettre à jour l'affichage en fonction des réponses du serveur. En revanche, la découverte complète des serveurs sur le réseau local reste partielle. De plus, certains aspects de l'intégration graphique restent encore simplifiés, notamment pour l'initialisation locale des parties réseau qui est surtout pensée pour le cas de deux joueurs. Il est donc plus juste de dire que la fonctionnalité réseau est satisfaisante dans l'ensemble, tout en conservant quelques points d'amélioration.

2.2 Fonctionnalité 2 AI :

2.2.1 Introduction et organisation du module IA

L'intelligence artificielle est la composante du projet qui cherche le meilleur coup à jouer selon une situation du jeu. Quoridor est un jeu où chaque décision peut avoir des conséquences bien plus loin dans la partie, comme par exemple la pose d'un mur censé bloquer l'adversaire et qui finalement se retrouve à nous bloquer nous. L'évaluation d'une position sera complexe car il faut donc prévoir à l'avance si le coup que nous jouons peut être désavantageux pour nous des dizaines de coups plus tard.

Pour cela, nous avons implémenté plusieurs stratégies d'intelligence artificielle de niveaux croissants. Ces stratégies sont organisées dans le package fr.bordeaux.ai, lui-même subdivisé en plusieurs sous-packages reflétant la séparation des responsabilités :

— fr.bordeaux.ai.base — les abstractions communes à toutes les IA
(AiPlayer, HeuristicEvaluator)
— fr.bordeaux.ai.random — le joueur aléatoire
— fr.bordeaux.ai.heuristics — les trois fonctions d'évaluation de Minimax
— fr.bordeaux.ai.minimax — l'algorithme Minimax et son builder
— fr.bordeaux.ai.mcts — l'algorithme Monte Carlo Tree Search
— fr.bordeaux.ai.nn — le modèle de réseau de neurones
Toutes les stratégies partagent une base commune : la classe abstraite

AiPlayer, qui hérite elle-même de Player. Cette classe impose à chaque IA d'implémenter une seule méthode centrale : `getNextMove(GameState state)`. C'est le point d'entrée unique par lequel le moteur de jeu demande à l'IA de choisir son prochain coup. Du point de vue du moteur, peu importe que l'IA soit aléatoire ou basée sur un réseau de neurones ; elle répond toujours via cette même méthode. C'est le principe du polymorphisme en programmation orientée objet.

Toute classe implémentant l'interface `HeuristicEvaluator` doit être capable d'analyser un état de jeu donné pour lui assigner une valeur ou un score numérique. Elle impose que toutes les heuristiques aient une méthode

`evaluate(GameState state)`. Cette interface permet de rendre les fonctions d'évaluation interchangeables à l'exécution, sans modifier le code de l'algorithme de recherche. C'est le patron de conception Strategy : on peut changer de stratégie d'évaluation comme on changerait de règle ; l'algorithme qui utilise cette règle n'a pas besoin de savoir laquelle est active. Par conséquent, l'interface Strategy est représentée par `HeuristicEvaluator`. Les classes `AdvancedHeuristic`, `IntermediateHeuristic` et `SimpleHeuristic` sont les implémentations concrètes de cette stratégie. Enfin, `MiniMaxPlayer` agit comme le contexte qui utilise la stratégie définie par `HeuristicEvaluator`.

Le Joueur Aléatoire (RandomPlayer) : Un Acteur Fondamental Avant d'explorer des algorithmes plus avancés, définissons le RandomPlayer. Il identifie tous les coups légaux possibles et en choisit un au hasard, avec une probabilité égale pour chaque option. Il est utilisé si MiniMax est interrompu par manque de temps pour atteindre la profondeur définie en paramètre.

Théorie approfondie de l'algorithme Minimax et élagage alpha-bêta

L'algorithme Minimax : stratégie en IA pour les jeux L'algorithme Minimax est une méthode conçue pour déterminer la meilleure action dans les jeux à deux joueurs, déterministes et à information complète. Dans son implémentation, l'algorithme suppose que l'adversaire joue le meilleur coup à sa disposition. Il repose sur une analyse des possibilités de jeu, structurée sous la forme d'un arbre de décision (Fig. 4).

Structure de l'arbre de jeu Minimax

1. Racine : L'état initial du plateau.
2. Branches : Chaque nœud se ramifie en branches représentant les coups légaux disponibles.
3. Feuilles : Aux extrémités de l'exploration, une fonction d'évaluation heuristique attribue un score numérique à la position (avantage pour le joueur IA, le joueur MAX).

Principe d'alternance (MAX et MIN) L'exploration de l'arbre alterne entre deux objectifs :
— Nœuds MAX (Tour de l'IA) : L'IA cherche à MAXimiser son score. Elle choisit le coup menant à l'état enfant avec la valeur heuristique la plus élevée.
— Nœuds MIN (Tour de l'Adversaire) : L'algorithme suppose que l'adversaire cherche à MINimiser le score de l'IA. Il simule le choix de l'adversaire menant à l'état enfant avec le score le plus bas.
Ce processus de remontée des valeurs (des feuilles à la racine) permet à l'IA de choisir le premier coup garantissant le meilleur résultat possible.

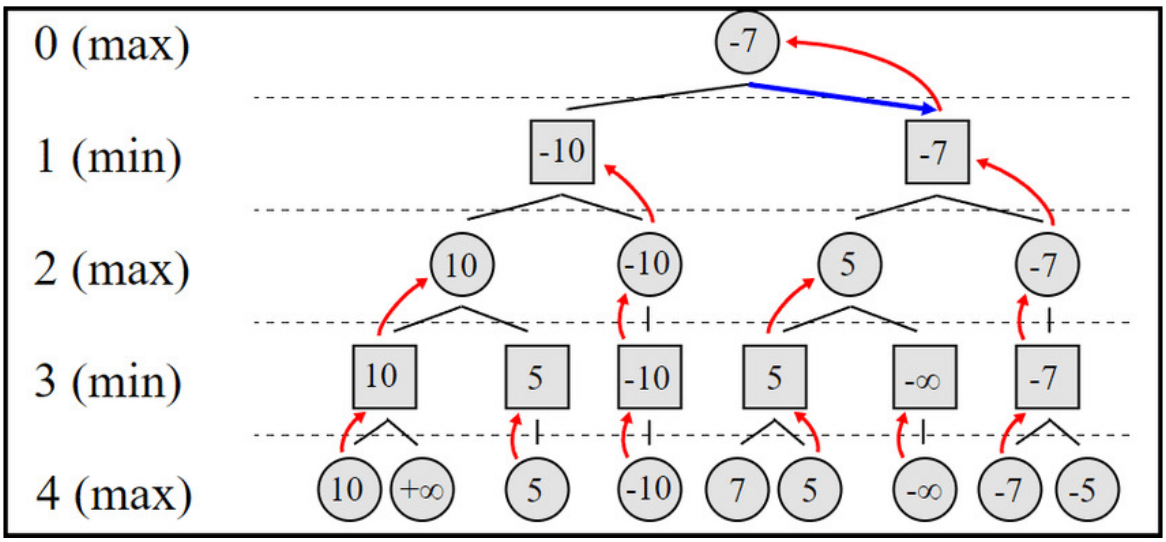


FIGURE 4 – Algorithme Minimax et arbre de décision

La complexité exponentielle du Minimax l'oblige à explorer des branches dont le calcul s'avère en réalité inutile, l'élagage Alpha-Bêta permet précisément de les ignorer.

2.2.2 Optimisation : L'Élagage Alpha-Bêta

L'élagage Alpha-Bêta est l'optimisation mathématique de référence pour le Minimax. Cette technique réduit le nombre de nœuds à explorer en ignorant les branches qui ne peuvent pas aboutir à un meilleur score que celui déjà garanti par une autre branche. L'élagage maintient l'optimalité du résultat avec un gain de vitesse considérable.

L'exploration exhaustive de l'arbre de jeu étant trop coûteuse en ressources, l'algorithme maintient deux bornes de coupe (des limites mathématiques permettant de stopper prématurément le calcul) :

— α : Le score minimum garanti pour le joueur MAX (l'intelligence artificielle).
— β : Le score maximum garanti pour le joueur MIN (l'adversaire).

Durant la descente dans l'arbre, l'algorithme compare constamment les options des deux joueurs, et l'exploration d'une branche est immédiatement interrompue (élaguée) dès que la condition $\alpha \geq \beta$ est validée. Pour comprendre logiquement cette mécanique, il faut considérer la valeur α comme l'assurance de l'intelligence artificielle (le joueur MAX) : c'est le score minimum qu'elle s'est déjà garanti en explorant une branche précédente. Lorsque l'algorithme commence à évaluer une nouvelle alternative (un autre coup possible pour MAX), il simule les réponses de l'adversaire (le joueur MIN). Si, au cours de cette simulation, l'algorithme découvre un coup pour MIN qui fait chuter l'évaluation à une valeur β inférieure ou égale à l'assurance α, le calcul s'arrête. En effet, le but de MIN étant de minimiser le score, il choisira à coup sûr cette réponse punitive, ou une pire encore s'il en trouve une en cherchant plus loin. Anticipant cela, MAX sait pertinemment que s'engager dans cette direction le forcera à subir ce score β, ce qui est strictement moins avantageux que son plan de secours α déjà sécurisé ailleurs. MAX ne choisira donc jamais d'emprunter ce chemin initialement. Par conséquent, il devient mathématiquement inutile de gaspiller du temps de calcul pour évaluer le reste de la branche, justifiant ainsi la coupe immédiate du nœud.

Cette coupe radicale permet de dédier la puissance de calcul aux branches viables, augmentant mécaniquement la profondeur d'analyse dans la limite de temps impartie.

2.2.3 L'Évaluation Heuristique : Le Moteur de Décision de l'IA

Lorsqu'un nœud atteint la profondeur maximale ou que le temps est écoulé, la partie n'est généralement pas terminée. L'algorithme fait alors appel à une fonction d'évaluation (heuristique) pour estimer la valeur tactique de cette position précise. Plus ce score est élevé, plus la situation est jugée favorable pour l'IA (MAX). Le projet propose trois niveaux d'évaluation :

L'Heuristique Simple (SimpleHeuristic) L'Heuristique Simple (SimpleHeuristic) se concentre sur l'objectif le plus rudimentaire du jeu, qui est d'avancer vers l'arrivée. D'un point de vue théorique, elle s'appuie sur une évaluation spatiale purement absolue en calculant le nombre de cases franchies en ligne droite par rapport à sa position de départ. Bien que cette méthode arithmétique soit extrêmement rapide à calculer, elle présente des limites tactiques majeures : elle reste totalement ignorante de son environnement puisqu'elle ne prend en compte ni les murs posés sur le plateau, ni la position actuelle de l'adversaire.

L'Heuristique Intermédiaire (MediumHeuristic) L'Heuristique Intermédiaire (MediumHeuristic) affine cette première approche en introduisant la notion d'avantage relatif. Plutôt que d'évaluer l'intelligence artificielle de manière isolée, elle reprend exactement la logique de l'heuristique simple et l'applique aux deux joueurs pour en extraire un différentiel. L'algorithme calcule le nombre de cases franchies en ligne droite par l'IA depuis sa position de départ, puis y soustrait cette même mesure effectuée pour l'adversaire. Cette évaluation reste strictement spatiale et calcule une distance directe, ignorant la présence éventuelle de murs sur le trajet. Un score positif indique ainsi clairement que l'IA est physiquement en tête de la course. Surtout, cette comparaison de distances incite naturellement l'algorithme à jouer de manière défensive. Si la pose d'un mur ralentit fortement la progression ennemie sans trop pénaliser celle de l'IA, l'écart mathématique entre les deux joueurs augmentera, validant ainsi la pertinence de ce blocage tactique.

L'Heuristique Avancée (AdvancedHeuristic) L'Heuristique Avancée (AdvancedHeuristic) constitue le véritable cerveau tactique du joueur Minimax. Elle calcule son score final en agrégeant plusieurs métriques dynamiques, c'est-à-dire des mesures chiffrées qui évoluent en temps réel selon la configuration du plateau, contrairement à des valeurs fixes. Ces métriques incluent l'analyse spatiale des obstacles, la gestion de la réserve de murs et la capacité à forcer des détours.

Tout d'abord, contrairement aux heuristiques précédentes basées sur des coordonnées brutes, elle effectue une analyse topographique concrète : elle étudie le relief du terrain créé par les murs. Elle calcule la vraie distance requise pour gagner en tenant compte du labyrinthe formé par les obstacles déjà posés, en utilisant l'algorithme de parcours en largeur BFS (Breadth-First Search). Ce dernier explore systématiquement toutes les cases accessibles pour garantir le calcul du chemin le plus court.

Ensuite, elle intègre une gestion des ressources en comparant le nombre de murs non joués restants dans la main de l'IA par rapport à ceux de l'adversaire. Cet avantage matériel est modulé par un coefficient fluctuant (wallCoeff). Ce coefficient traduit le fait qu'un mur conservé en réserve en début de partie rapporte beaucoup de points en raison de son fort potentiel de blocage, mais que sa valeur tactique (son utilité réelle pour gagner) diminue à mesure que l'IA approche de la victoire et que les opportunités de pose se raréfient.

Le cœur de cette heuristique repose sur la stratégie de contournement, c'est-à-dire l'art d'obliger l'adversaire à s'éloigner de son chemin le plus direct. Pour évaluer la réussite de cette stratégie, l'algorithme calcule un bonus de détour en comparant la distance théorique de l'adversaire sans aucun obstacle (la distance de Manhattan, qui est la somme des différences absolues de coordonnées) avec sa distance réelle trouvée par le parcours BFS. La différence entre ces deux valeurs représente le détour exact imposé par les murs placés sur le plateau. L'IA est ainsi lourdement

récompensée (WALL_BONUS) si elle parvient à créer des goulots d'étranglement obligeant l'adversaire à emprunter de longs chemins sinueux.

Enfin, l'algorithme gère les conditions de victoire de manière prioritaire pour éviter que des calculs secondaires ne masquent un gain ou une perte imminente. Dans l'implémentation, le seuil de victoire est une constante nommée WIN_THRESHOLD fixée à la valeur 1, représentant la distance minimale, calculée en nombre de pas par l'algorithme BFS, séparant un pion de sa ligne d'arrivée. Ce seuil est déterminé dynamiquement à chaque évaluation : l'IA lance un parcours de graphe pour elle-même et pour son adversaire afin de connaître le nombre précis de cases restant à parcourir en tenant compte des murs. Si le résultat est inférieur ou égal à 1, cela signifie que le joueur est soit déjà sur sa ligne d'arrivée, soit à une seule case de celle-ci. Dès que cette proximité extrême est détectée, la fonction interrompt immédiatement l'agrégation des autres métriques, comme la réserve de murs ou les bonus de détour, car elles n'ont plus d'importance tactique face à une fin de partie immédiate. L'algorithme renvoie alors un score absolu de 10 000 pour une victoire de l'IA ou de -10 000 pour une victoire adverse, garantissant que l'intelligence artificielle privilégiera toujours le coup menant au succès final plutôt qu'une simple amélioration stratégique.

Structures de Données et Optimisations (Minimax) L'implémentation repose sur des structures de données spécifiquement choisies pour limiter l'empreinte mémoire (la quantité de mémoire vive RAM consommée) et accélérer la vitesse d'exécution :

- File à double entrée (Deque) pour l'exploration de l'arbre : Afin d'éviter l'instanciation massive (la création répétitive et très coûteuse de nouveaux objets en mémoire pour chaque scénario), la classe MinimaxPlayer ne clone pas le plateau de jeu pour chaque branche. Elle maintient l'historique de la branche courante dans une structure de pile (Deque<Move>). L'algorithme procède par mutation directe (modification de l'état unique du plateau) grâce à un mécanisme de backtracking (retour sur ses pas) : on simule un coup pour descendre dans l'arbre, puis on le dépile pour l'annuler et remonter.
- Interface Fonctionnelle (BiFunction) : Plutôt que de vérifier à chaque coup quel algorithme utiliser via une structure de contrôle classique (type if/else), le constructeur de MinimaxPlayer fige le comportement de l'IA dès son initialisation. Il assigne la méthode de résolution choisie — Minimax standard ou Approfondissement Itératif — à un attribut nommé mode. Lors de l'exécution, l'IA appelle directement cette fonction stockée, ce qui simplifie le flux de calcul en éliminant les branchements logiques répétitifs au cœur de la prise de décision.
- Table de hachage sécurisée (ConcurrentHashMap) : Les paramètres internes de l'IA (profondeur, temps, heuristique) sont exportés via la méthode getSettings à l'aide d'une ConcurrentHashMap (un dictionnaire stockant des paires clé-valeur). Cette structure est spécifiquement conçue pour être thread-safe (protégée contre les crashs si plusieurs processus ou éléments de l'interface graphique tentent de la lire en même temps).
- Listes dynamiques (List) pour le calcul heuristique : Dans la classe AdvancedHeuristic, l'évaluation tactique ne s'arrête pas aux coordonnées brutes. Le calcul des distances réelles s'appuie sur une liste pour stocker la séquence des identifiants des cases validées par l'algorithme BFS (Breadth-First Search, un parcours de graphe en largeur garantissant de trouver le chemin le plus court tout en contournant les murs existants).

L'Approfondissement Itératif (Iterative Deepening) Cet algorithme, plutôt que d'explorer l'arbre à une profondeur fixe définie arbitrairement, exécute des recherches successives à des profondeurs croissantes. Le principal atout de cette méthode est sa capacité à fournir une réponse optimale à tout moment. Si le temps alloué (reflexionTime) s'écoule pendant l'exploration de la profondeur p, l'algorithme interrompt le calcul et renvoie le meilleur coup identifié lors de l'itération à la profondeur $p-1$. Contrairement à Minimax qui, s'il est interrompu, fournit un résultat aléatoire, l'Iterative Deepening garantit une décision tactique basée sur l'exploration complète la plus profonde possible. Cette méthode introduit donc une stratégie de gestion dynamique de la profondeur de recherche qui vient compléter l'élagage Alpha-Bêta pour pallier l'imprévisibilité de la qualité du coup joué.

Cependant, l'inconvénient de cette méthode réside dans la redondance : chaque nouvelle itération recommence l'exploration des premiers niveaux de l'arbre déjà parcourus. Cependant, dans un jeu comme le Quoridor où le facteur de branchement est important, ce surcoût reste acceptable car le temps recalculé pour une profondeur p va représenter un temps de calcul très faible par rapport à celui de l'itération finale.

Bien que l'algorithme Minimax soit redoutable, son exploration systématique montre ses limites face à des situations présentant un fort facteur de branchement, où le nombre de coups possibles devient beaucoup trop grand pour être calculé dans le temps imparti. C'est pour pallier cette explosion combinatoire qu'intervient l'algorithme Monte Carlo Tree Search (MCTS).

Mécanique de l'Algorithme Monte Carlo Tree Search Contrairement au Minimax qui tente d'évaluer toutes les éventualités, le MCTS ne vérifie pas toutes les possibilités. Il s'agit d'une approche probabiliste, signifiant qu'elle fait intervenir l'aléatoire dans ses choix de simulation, et asymétrique, car elle concentre sa puissance de calcul pour explorer en profondeur les branches les plus prometteuses plutôt que de parcourir l'arbre de manière uniforme[5]. Le processus itératif s'exécute en boucle selon quatre phases distinctes orchestrées par la méthode getBestMoveAfterSearch dans la classe MctsNode (Fig. 5).

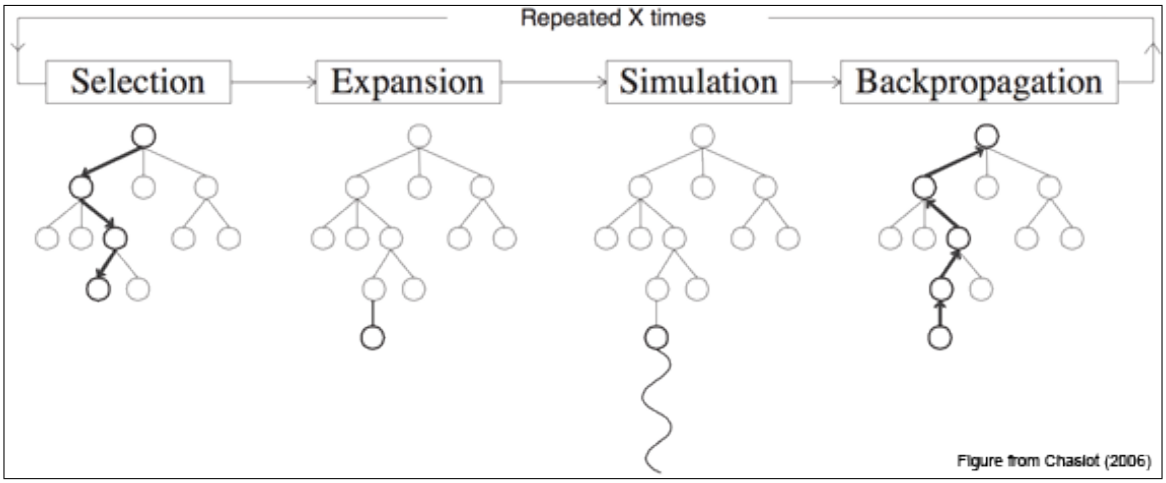


FIGURE 5 – Les quatre phases de l'algorithme MCTS

La première phase est la sélection, étape au cours de laquelle l'algorithme amorce sa recherche en descendant dans l'arbre depuis la racine. Le but de cette descente est de trouver le candidat idéal pour poursuivre l'exploration. L'algorithme obéit à une règle de priorité stricte dictant le passage à l'étape suivante. En effet, tant qu'un état a déjà testé l'intégralité de ses coups possibles et possède des nœuds enfants, la phase de sélection continue de s'enfoncer dans l'arbre. À chaque intersection, l'intelligence artificielle choisit de descendre vers le nœud enfant le plus "prometteur", c'est-à-dire celui qui maximise le score évalué par la méthode bestUCT. Ce score prometteur désigne le meilleur compromis mathématique calculé entre l'exploitation d'une branche affichant déjà un fort taux de victoire et l'exploration nécessaire des chemins encore peu visités, principe fondamental du MCTS[5].

Cependant, dès que l'algorithme croise au cours de sa descente un nœud disposant encore d'au moins un coup légal non essayé (une condition vérifiée dynamiquement par la méthode hasUntriedLegalMoves), la phase de sélection s'arrête net. Ce nœud partiellement exploré devient alors la cible prioritaire absolue pour la phase suivante, garantissant ainsi que l'arbre s'élargisse systématiquement pour évaluer toutes les options initiales d'une position avant de se creuser aveuglément en profondeur.

Vient ensuite la phase d'expansion. Si le nœud sélectionné n'est pas un état de fin de partie, un coup non testé est extrait pour créer un nouveau nœud enfant dans l'arbre de recherche. Pour implémenter cela efficacement, la classe MctsNode s'appuie sur des listes dynamiques. Lors de sa création, chaque état génère et stocke ses coups potentiels dans une liste dédiée nommée untriedMoves. Afin de garantir la stochasticité de l'algorithme, c'est-à-dire l'introduction d'un hasard mathématique visant à neutraliser l'ordre prévisible dans lequel le jeu génère initialement les actions, cette collection est immédiatement mélangée grâce à la fonction Collections.shuffle[5].

Une optimisation majeure de cette phase d'expansion réside dans la vérification paresseuse des coups. Dans un jeu comme Quoridor, calculer la légalité absolue de toutes les poses de murs possibles est extrêmement lourd en ressources. Pour éviter ce goulet d'étranglement, la création du nœud génère uniquement des coups dits "pseudo-légaux". Ce type d'optimisation est couramment utilisé dans les implémentations pratiques du MCTS[7].

Dans le cas contraire, si l'expansion aboutit à un état en cours de jeu, l'algorithme enclenche la troisième phase : la simulation (Playout). À partir de ce nouveau nœud, l'algorithme simule une fin de partie en jouant une série de déplacements aléatoires jusqu'à déterminer un vainqueur.

Enfin, la rétropropagation (Backpropagation) clôture le cycle en remontant le résultat de cette simulation le long de la branche explorée afin de mettre à jour les statistiques des nœuds parents. Chaque nœud conserve un nombre de visites et un nombre de victoires. Ces valeurs permettent de calculer le taux de victoire utilisé dans la formule UCT pour guider les futures explorations.

Exploration et Exploitation : Le Calcul UCT La véritable intelligence du MCTS réside dans sa phase de sélection, pilotée par la classe MctsPlayer. Bien que cette classe puisse dynamiquement déléguer ce choix à un réseau de neurones artificiels (une approche détaillée dans la section suivante), nous nous concentrerons ici sur sa méthode d'évaluation classique nommée selectBestChildUsingUCT. Pour choisir le chemin à emprunter dans l'arbre, l'algorithme utilise la formule mathématique UCT (Upper Confidence bounds applied to Trees). Cette équation garantit un compromis rigoureux entre l'exploitation et l'exploration.

Mathématiquement, l'algorithme calcule pour chaque nœud enfant un score composé de deux parties. La première partie est le taux de victoire brut, obtenu en divisant le nombre de victoires par le nombre de visites de ce nœud. Cela représente l'exploitation, signifiant concrètement le fait de privilégier les coups ayant déjà prouvé leur efficacité. La seconde partie est le terme d'exploration, calculé en multipliant une constante fixée à 1.41 par la racine carrée du

logarithme des visites du nœud parent divisé par les visites de l'enfant. Ce terme d'exploration augmente artificiellement le score des branches qui ont été peu visitées. Ainsi, l'algorithme est naturellement poussé à vérifier les coups ignorés pour s'assurer de ne rater aucune stratégie cachée, tout en continuant d'approfondir les lignes de jeu majoritairement gagnantes.

Les Limites de la Simulation Aléatoire et l'Introduction au Machine Learning Si la méthode UCT standard est très performante pour diriger la recherche, elle repose fondamentalement sur la phase de simulation (le Playout aléatoire) pour évaluer la qualité d'un nœud. Dans un jeu complexe, jouer des coups au hasard jusqu'à la fin de la partie peut aboutir à des résultats chaotiques qui ne reflètent pas la véritable valeur stratégique d'une position. Un simple mur placé au hasard peut fausser toute l'évaluation. Pour pallier cette limite, l'intelligence artificielle introduit un modèle d'apprentissage automatique supervisé, une architecture héritée des travaux révolutionnaires d'AlphaZero combinant recherche arborescente et évaluation neuronale[8]. La classe `MctsPlayer` agit alors comme un aiguilleur paramétrable offrant la possibilité de configurer le moteur de recherche selon deux modes exclusifs, c'est-à-dire en utilisant soit l'approche UCT traditionnelle, soit le réseau de neurones. Lorsque l'apprentissage automatique est sélectionné, le modèle neuronal évalue instantanément les chances de victoire de la position en cours au lieu de se fier au calcul mathématique classique. Cette approche remplace ainsi avantageusement la dépendance à la lourde phase de simulation aléatoire par une prédiction déterministe et stratégiquement éclairée. Le terme déterministe certifie que pour une configuration strictement identique du plateau, le réseau de neurones renverra toujours exactement le même score de victoire, puisqu'il remplace le hasard des simulations par un calcul matriciel aux paramètres fixes. Parallèlement, cette prédiction est qualifiée de stratégiquement éclairée car elle ne se base plus sur des déplacements effectués à l'aveugle. Le modèle analyse concrètement la qualité de la position en évaluant la véritable distance séparant les joueurs de leur ligne d'arrivée et en comptabilisant le nombre de murs restants dans les réserves, appliquant ainsi une réelle compréhension tactique acquise lors de son entraînement.

L'Architecture du Neurone et la Gestion des Tenseurs L'évaluation neuronale est encapsulée dans la classe NeuralNetworkEvaluator, qui sert d'interface avec le moteur de calcul TensorFlow. Ce modèle s'appuie spécifiquement sur une architecture de type régression logistique, ce qui correspond structurellement à l'utilisation d'un unique neurone artificiel.

Le fonctionnement de ce neurone repose sur une équation mathématique fondamentale où la prédiction finale est le résultat de l'opération $P = \sigma(W \cdot X + b)$ (Fig. 6). Dans cette formule, la variable X représente les caractéristiques en entrée fournies par le jeu. La variable W désigne les poids de la matrice, représentant l'importance accordée par l'intelligence artificielle à chaque paramètre. Enfin, la variable b correspond au biais, un ajustement numérique global indépendant des entrées.

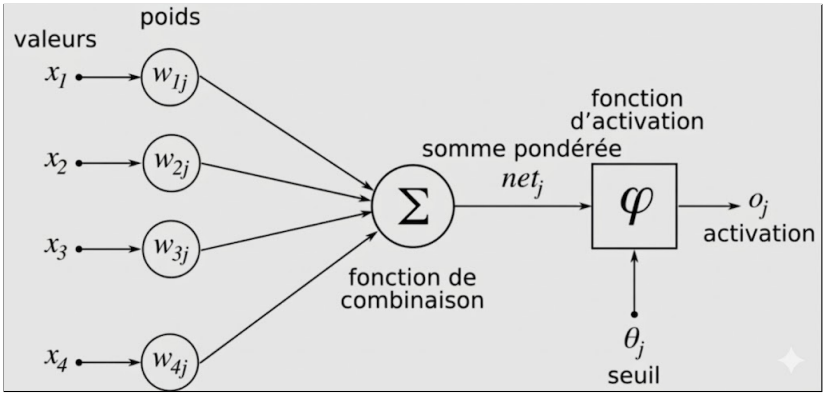


FIGURE 6 – Modèle de neurone artificiel pour l'évaluation

Le système prend en entrée quatre caractéristiques mathématiques précises extraites de la partie en cours. Il calcule d'abord la véritable distance séparant le joueur de sa ligne d'arrivée en effectuant un parcours de graphe BFS. Pour que cette donnée soit exploitable par le réseau de neurones, elle doit être normalisée. L'algorithme divise donc ce nombre de pas par la distance maximale théorique du plateau, puis inverse mathématiquement ce résultat. Ainsi, une distance nulle produit la valeur maximale de 1.0 pour indiquer une victoire atteinte, tandis que l'éloignement maximal donne un résultat de 0.0. Il intègre ensuite la distance de l'adversaire, elle aussi obtenue par un algorithme BFS et divisée par la distance maximale. Il est important de noter que cette valeur adverse n'est pas inversée, car pour l'intelligence artificielle, un résultat de 1.0 indiquant que l'ennemi est bloqué très loin constitue une situation hautement favorable. Les troisième et quatrième paramètres sont respectivement le nombre de murs restants pour l'IA et pour son adversaire, tous deux divisés par leur capacité initiale de dix (si la capacité initiale est supérieure à 10, on s'assure de rester dans l'intervalle [0,1] en prenant 1.0 au maximum) . Cette dernière opération permet de ramener

l'intégralité des données fournies au neurone sur une échelle mathématique parfaitement homogène comprise entre zéro et un.

L'algorithme de TensorFlow applique alors une multiplication matricielle entre ces quatre entrées X et leurs poids respectifs W, additionne le biais b, puis fait passer le résultat brut par la fonction d'activation sigmoïde désignée par le symbole σ. Cette fonction prend la forme d'une courbe en "S" écrasant les valeurs numériques pour forcer un résultat strictement compris entre 0 et 1, représentant ainsi de façon lisible le pourcentage direct de chances de victoire de la position évaluée.

Techniquement, pour interagir avec le cœur natif de TensorFlow, les données primitives Java sont systématiquement converties en tenseurs de type TFloat32. Ce sont des matrices multidimensionnelles hautement optimisées pour être manipulées très rapidement par le moteur C++ sous-jacent. De plus, le graphe computationnel modélisant ces opérations mathématiques et la session d'exécution sont maintenus ouverts en mémoire vive. Cette architecture évite le coût de latence massif qu'impliquerait une recompilation du modèle à chaque nouvelle inférence requise par le MCTS. Pour gérer rigoureusement ces ressources externes, la classe implémente l'interface native AutoCloseable. Cette sécurité garantit la destruction explicite des objets alloués en C++ à la fermeture du programme, prévenant toute fuite de mémoire (Memory Leak) puisque ces objets échappent par nature au ramasse-miettes (Garbage Collector) de Java.

L'Entraînement Supervisé et la Descente de Gradient Pour que ce neurone artificiel et ses poids internes soient capables d'évaluer correctement une position, il faut d'abord les ajuster lors d'une phase d'apprentissage autonome. La classe NeuralNetworkTrainer est responsable de cet entraînement supervisé. Elle génère d'énormes quantités de données en faisant s'affronter l'IA contre elle-même (Self-Play) au travers de milliers de parties simulées. Durant ces simulations de génération de données, l'algorithme applique une politique d'exploration appelée Epsilon-Greedy. Une probabilité fixée à dix pour cent force ponctuellement l'IA à jouer des coups totalement aléatoires. Le reste du temps, le système ne cherche pas à calculer le meilleur mouvement tactique absolu de la position, mais délègue plutôt la décision à un algorithme Minimax intentionnellement bridé à une profondeur de recherche d'un seul tour. Ce choix technique est un compromis indispensable pour garantir la rapidité des simulations, puisqu'une recherche complète et profonde prendrait un temps incalculable pour générer les milliers de parties requises par l'apprentissage. Cette mécanique hybride empêche finalement l'algorithme de tourner en boucle sur les mêmes séquences d'ouverture et le force à découvrir de nouvelles configurations complexes du labyrinthe tout en générant des données de jeu cohérentes.

Une fois ces parties terminées et les résultats finaux connus, le réseau compare ses propres prédictions passées avec la réalité des victoires et des défaites. Cette différence est mesurée avec une grande précision par une fonction mathématique nommée Entropie Croisée Binaire, ou Binary Crossentropy. Cette fonction de perte calcule l'écart entre la probabilité prédite par la sigmoïde et le label réel de la partie, valant 1.0 pour une victoire observée et 0.0 pour une défaite. La particularité mathématique de l'Entropie Croisée Binaire est d'appliquer une pénalité logarithmique extrêmement sévère si le modèle se trompe en étant très confiant, par exemple s'il prédisait 90 pour cent de chances de victoire pour une position qui a finalement mené à un échec.

Afin de corriger cette erreur de prédiction, l'algorithme fait appel à un optimiseur basé sur la méthode de la Descente de Gradient. Ce mécanisme mathématique calcule la dérivée de l'erreur pour déterminer dans quelle direction modifier les variables internes. Il ajuste alors de manière infime les poids synaptiques W et le biais b du neurone en fonction d'un taux d'apprentissage défini, afin que lors de la prochaine situation similaire, la prédiction se rapproche naturellement de la réalité. En répétant ce cycle d'évaluation et de correction sur des dizaines de milliers d'exemples répartis sur plusieurs centaines d'époques d'entraînement, le modèle met progressivement à jour sa matrice de poids pour affiner sa compréhension tactique du jeu.

3 Architecture du projet

3.1 Justificatif de l'architecture

Notre projet consiste à implémenter le jeu Quoridor en Java, en respectant strictement l'ensemble des règles détaillées dans la section précédente « Les règles du jeu ». Au-delà de la simple reproduction des mécaniques, notre cahier des charges impose plusieurs contraintes et objectifs qui influencent directement le choix de l'architecture logicielle. En particulier, nous devons intégrer trois modes d'interaction distincts : le mode en ligne de commande (CLI), une interface graphique (GUI) et un mode réseau permettant de jouer avec d'autres utilisateurs à distance. À cela s'ajoutent des exigences concernant l'intelligence artificielle : le système doit pouvoir exécuter plusieurs algorithmes, notamment Minimax et Monte Carlo Tree Search (MCTS), pour simuler des adversaires compétitifs et variés.

Pour répondre efficacement à ces besoins complexes, nous avons choisi une architecture modulaire orientée domaine, inspirée de la Clean Architecture telle que décrite par Martin[9]. Cette décision repose sur plusieurs considérations pratiques et théoriques, toutes centrées sur la nécessité de séparer clairement les responsabilités et de rendre notre

application à la fois extensible, maintenable et testable.

Principes fondamentaux de la Clean Architecture La Clean Architecture repose sur un principe central : la séparation stricte entre la logique métier et les détails techniques de l'implémentation. Dans notre contexte, cela signifie que toutes les règles et entités du jeu — pions, murs, plateaux, mouvements, validation des coups, état de la partie — sont encapsulées dans un cœur métier indépendant de toute interface utilisateur, du réseau ou de l'intelligence artificielle. Ce cœur, que nous avons regroupé dans le package `Core`, contient toutes les entités et services nécessaires au fonctionnement du jeu. Il ne connaît rien des mécanismes de présentation, de stockage ou de communication, ce qui garantit que la logique métier reste cohérente et isolée.

Cette approche est cohérente avec les recommandations de Fowler[10] sur les patterns d'architecture logicielle. Fowler insiste sur la nécessité de séparer les préoccupations et de structurer les systèmes en couches bien définies, afin de réduire les dépendances entre les composants et de faciliter la maintenance. En suivant ce principe, notre architecture repose sur des couches clairement identifiées :

- **Cœur métier (`Core`)** : contient toutes les entités métier, les règles de validation, les calculs de victoire et les états de partie.
- **Interfaces d'adaptation (`Adapters`)** : représentent les points d'entrée et de sortie du système, tels que la CLI, la GUI, le réseau et les modules d'IA.
- **Infrastructure (`Infrastructure`)** : regroupe les implémentations concrètes nécessaires pour faire communiquer le cœur métier avec les interfaces, comme la persistance ou les modules de communication réseau.

Avantages pratiques de cette approche L'un des principaux avantages de cette architecture est sa modularité. En séparant clairement le cœur métier des interfaces, nous pouvons :

- Développer et tester chaque composant de manière indépendante. Les tests unitaires peuvent se concentrer uniquement sur le cœur métier sans nécessiter l'exécution de la GUI ou du réseau.
- Ajouter de nouvelles fonctionnalités ou interfaces sans impacter les modules existants. Par exemple, il est possible de créer une nouvelle version web ou mobile de Quoridor simplement en développant un nouvel adaptateur qui interagit avec le cœur via des interfaces abstraites.
- Introduire ou modifier les algorithmes d'intelligence artificielle (Minimax, MCTS, ou d'autres) sans toucher aux règles du jeu. Chaque algorithme peut être implémenté comme un service indépendant qui sollicite le cœur métier pour connaître l'état de la partie et proposer un coup.

Un autre avantage majeur réside dans la robustesse et la sécurité fonctionnelle. Toutes les actions initiées par un joueur ou par un algorithme passent par le cœur métier pour validation. Cela signifie qu'aucune interface ne peut créer d'état invalide : impossible de poser un mur illégal, de déplacer un pion hors du plateau ou de contourner les règles fondamentales. Cette approche centralise la logique critique et prévient les bugs liés à des incohérences entre différents modules.

Interaction entre couches et adaptateurs Les adaptateurs constituent le lien entre le monde extérieur et le cœur métier. Chaque adaptateur utilise des interfaces abstraites définies dans le cœur pour communiquer avec celui-ci. Par exemple :

- La CLI lit les entrées du joueur (déplacement de pion ou pose de mur), transmet ces actions au cœur métier via l'interface `GameEngine`, puis affiche le résultat sur le terminal.
- La GUI capte les clics de l'utilisateur sur le plateau graphique, envoie les actions au cœur, puis met à jour l'affichage en fonction de l'état retourné.
- Le module réseau convertit les messages reçus d'un autre joueur en actions valides, puis informe le cœur métier. De même, les changements d'état du cœur sont convertis en messages envoyés aux clients connectés.
- Les modules d'IA consultent le cœur métier pour analyser l'état du plateau et générer le meilleur coup possible selon la stratégie choisie.

Cette conception permet à chaque adaptateur de se concentrer sur ses responsabilités spécifiques sans se soucier des règles métier, tout en assurant une cohérence globale du système.

Testabilité et maintenance L'un des aspects les plus valorisés de cette architecture est sa testabilité. Le cœur métier peut être testé en isolation complète, ce qui permet de vérifier toutes les règles du jeu, la gestion des murs, la détection de victoire et les comportements spécifiques sans dépendre de l'interface graphique ou du réseau. Cela réduit considérablement les risques d'erreurs et simplifie la détection de bugs.

De plus, la maintenance est facilitée. Si, à l'avenir, nous souhaitons modifier l'interface graphique ou ajouter un mode de jeu supplémentaire, aucune modification n'est nécessaire dans le cœur métier. Les interfaces sont découplées, ce qui signifie que les changements techniques ou esthétiques ne perturbent pas la logique fondamentale du jeu. Cette modularité garantit également une meilleure évolutivité à long terme : le projet peut croître et intégrer de nouvelles fonctionnalités sans remettre en cause les composants existants.

Flexibilité pour les algorithmes d'intelligence artificielle Dans le cadre de notre projet, la Clean Architecture est particulièrement adaptée à l'intégration d'algorithmes d'IA. La séparation des couches permet d'implémenter

indépendamment :

— **Minimax**, pour évaluer systématiquement les coups et anticiper les réponses de l'adversaire dans un arbre de décision.
— **Monte Carlo Tree Search (MCTS)**, pour explorer de manière probabiliste les coups et identifier les stratégies les plus prometteuses.

Chaque algorithme interagit uniquement avec le cœur métier via des interfaces abstraites, ce qui permet d'ajouter ou de remplacer un algorithme sans modifier la logique du jeu. Par exemple, nous pourrions expérimenter un nouvel algorithme hybride combinant MCTS et Minimax, ou encore intégrer des techniques d'apprentissage automatique pour améliorer les performances de l'IA, sans toucher au reste de l'architecture.

En résumé, le choix d'une architecture modulaire orientée domaine, inspirée de la Clean Architecture[9], se justifie pleinement par les besoins spécifiques de notre projet Quoridor. Elle garantit la cohérence de la logique métier, facilite l'intégration de multiples interfaces, permet l'ajout et le test d'algorithmes d'intelligence artificielle variés, et assure une maintenance et une évolutivité sur le long terme. L'adoption des principes de séparation des responsabilités et de structuration en couches, également recommandée par Fowler[10], renforce cette cohérence et permet de construire un système robuste, flexible et modulaire, capable de répondre à tous les objectifs fonctionnels et techniques définis dans notre cahier des charges.

3.2 Vue d'ensemble des packages

Cette organisation en packages nous permet de structurer le projet de manière claire et cohérente. Chaque package a un rôle précis, ce qui favorise la modularité, la testabilité et la maintenance du code dans notre projet. De plus, elle nous facilite l'ajout futur de fonctionnalités, qu'il s'agisse d'IA avancées, de nouvelles interfaces ou de modes de jeu supplémentaires, tout en garantissant la robustesse et la cohérence des règles de Quoridor.

Voici les onze packages distincts de notre projet, chacun ayant une responsabilité bien définie que nous décrivons ici :

Package	Responsabilité
core	constitue le cœur métier de l'application. Il regroupe toutes les entités fondamentales du jeu, telles que `Board` pour représenter le plateau, `GameState` pour l'état général de la partie, `Player` pour les joueurs, `Move` pour les déplacements, `Position` pour la localisation des pions, ainsi que `GameEngine` et `RuleChecker` pour gérer les règles et la logique de progression du jeu. Ce package est totalement indépendant des interfaces graphiques, de la ligne de commande ou des modules réseau, ce qui garantit que toutes les règles sont appliquées de manière cohérente, quel que soit le mode de jeu choisi.
Main	contient le point d'entrée de l'application. Il est responsable de la résolution du mode de lancement, que ce soit en CLI, en GUI ou en réseau, et applique le patron *Factory* pour initialiser les composants nécessaires selon le contexte. Cette centralisation du lancement permet de simplifier la configuration initiale et de garantir que tous les modules sont correctement instanciés avant le démarrage de la partie.
cli	gère l'interface en ligne de commande. Il utilise le pattern *Command* pour représenter chaque action que l'utilisateur peut effectuer, qu'il s'agisse de déplacer un pion, de poser un mur ou de demander l'aide. Ce design permet de découpler les commandes de l'exécution effective et de faciliter l'ajout de nouvelles commandes sans modifier la logique existante.
gui	basé sur JavaFX, implémente l'interface graphique. Il propose des menus principaux tels que `File` et `Game`, et permet à l'utilisateur de définir ses propres raccourcis clavier pour chaque action. Cette flexibilité améliore l'expérience utilisateur et rend l'interface plus ergonomique, tout en séparant la présentation de la logique métier contenue dans le package `core`.
network	est dédié à la gestion des fonctionnalités réseau. Il implémente un serveur TCP capable de gérer plusieurs clients simultanément, soit via un thread par client, soit par programmation asynchrone. Il intègre également une découverte UDP pour faciliter la connexion automatique des clients au serveur. Ce package assure la communication entre joueurs distants et garantit que toutes les actions reçues sont validées par le cœur métier.
ai	regroupe les différentes intelligences artificielles implémentées dans le projet. Il inclut des IA simples aléatoires, des IA basées sur Minimax avec élagage alpha-bêta et *iterative deepening*, des IA utilisant Monte Carlo Tree Search, et des prototypes utilisant des réseaux de neurones. Ce package interagit uniquement avec le cœur métier via des interfaces abstraites, permettant d'échanger ou d'ajouter de nouveaux algorithmes facilement.
persistence	s'occupe de la sérialisation et désérialisation de l'état de jeu vers un fichier texte. Il assure que l'utilisateur peut sauvegarder et recharger une partie à tout moment, en conservant la cohérence des données avec le cœur métier.
util	contient des outils généraux, dont un *Logger* singleton utilisé par l'ensemble de l'application pour tracer les actions, les erreurs et les événements importants, facilitant ainsi le débogage et la maintenance.
i18n	gère la langue du logiciel. Il détecte automatiquement les variables d'environnement `LANG` ou `LC_ALL` pour déterminer la langue par défaut, et permet d'adapter l'affichage et les messages aux préférences de l'utilisateur.
graph	fournit une représentation du plateau sous forme de graphe, avec une liste d'adjacence. Il contient le `GraphBuilder` qui permet de modéliser dynamiquement les mouvements possibles et les blocages liés aux murs, facilitant ainsi les calculs pour les IA et la validation des coups.
config	s'occupe de la configuration globale du logiciel. Il lit un fichier `.quoridorrc` situé dans le répertoire personnel de l'utilisateur (HOME) pour appliquer les options par défaut. Si le fichier est absent, un fichier minimal est créé automatiquement. Ce package centralise toutes les préférences, évitant la duplication de la configuration dans différents modules.

TABLE 1 – Vue d'ensemble des packages

3.3 Vue en couches concentriques

Dans cette section, nous vous proposons notre architecture globale en couches concentriques. Les dépendances entre packages ne doivent pointer que vers l'intérieur du système. Le module **core** — qui contient les entités métier — ne connaît aucun autre module. Ce sont **network**, **gui** ou **cli** qui dépendent de **core**, jamais l'inverse. Cette contrainte est visible dans le diagramme.

Schéma en couches concentriques de notre architecture du projet (Fig. 7).

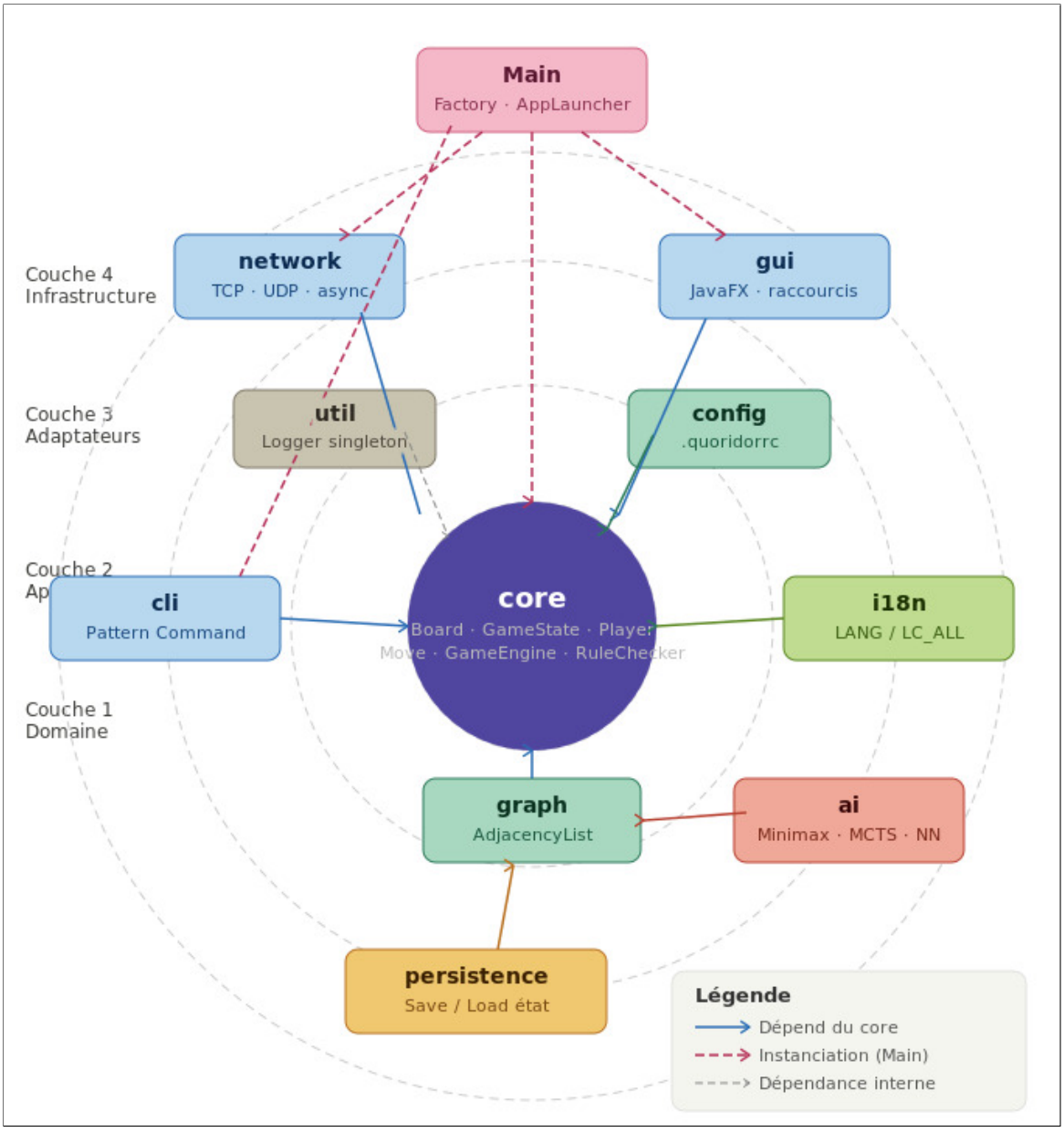


FIGURE 7 – Une architecture en couches concentriques

3.4 Couche Domaine — Package `core`

Le module **core** est le cœur du système. Il encapsule l'intégralité de la logique métier du jeu et n'a aucune dépendance externe. Le **RuleChecker** appelle le module **graph** pour vérifier qu'après la pose d'un mur, chaque joueur dispose encore d'un chemin vers son bord cible.

Voici un diagramme indiquant comment sont liées les différentes classes du package Core.

Schéma de la couche Core de notre architecture (Fig. 8).

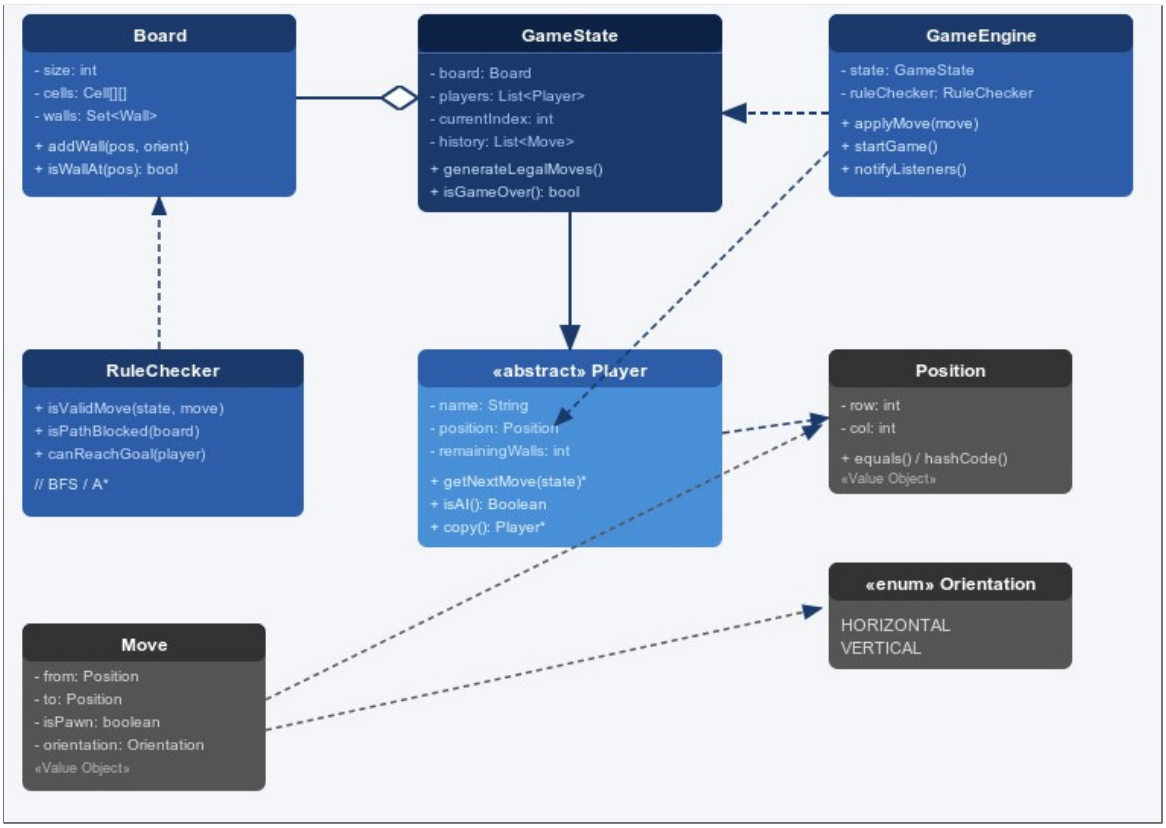


FIGURE 8 – Le cœur métier (Core)

3.5 Couche Principale — Package `Main`

Le module **Main** joue le rôle de **composition root** : il instancie tous les autres modules et câble leurs dépendances. Il utilise le pattern **Factory** via *AppLauncher* pour créer les adaptateurs selon le mode de lancement.

Au démarrage, le Main lit la configuration (.quoridorrc) pour déterminer le mode de jeu. Il instancie network si une partie réseau est demandée, gui pour le mode graphique, cli pour le mode terminal. Il injecte les dépendances dans chaque module sans que ceux-ci se connaissent mutuellement.

Voici un diagramme indiquant comment sont liées les différentes classes du package Main.

Schéma de la couche Main de notre architecture (Fig. 9).

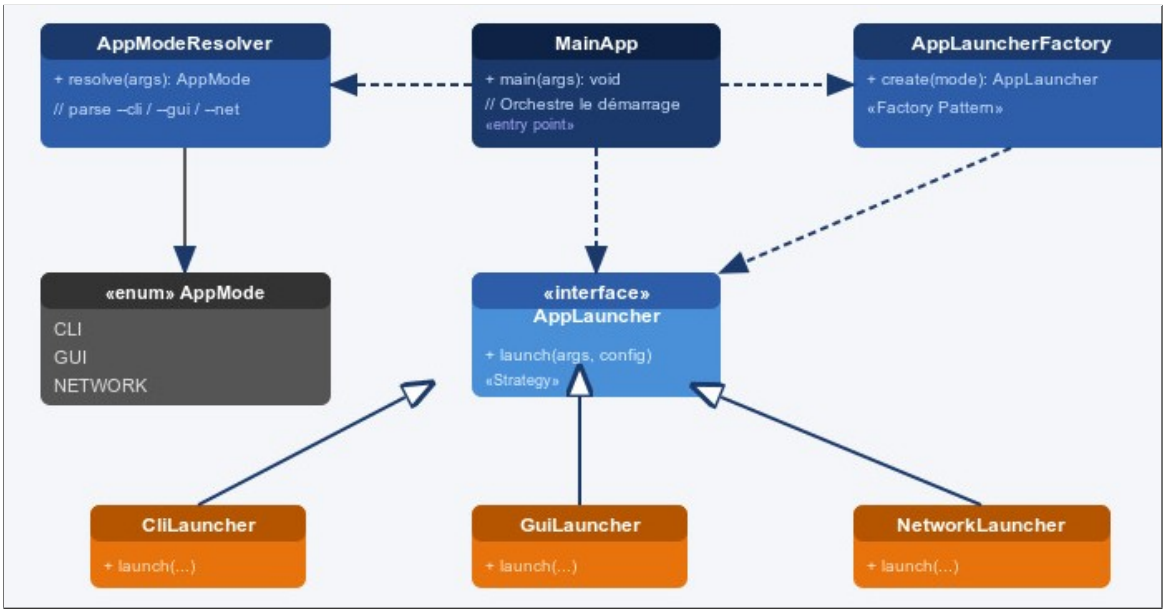


FIGURE 9 – Couche Principale (Main)

3.6 Interface CLI — Pattern Command

Le module **cli** implémente le pattern **Command** : chaque commande saisie est encapsulée dans un objet qui connaît son action et peut être annulée. Ce pattern est adapté à Quoridor car il permet de rejouer ou d'annuler l'historique des coups. Notre CLI est instanciée par Main au même titre que GUI et Network. Il lit les entrées console, les analyse (parse), construit les objets Command correspondants et les exécute sur *GameEngine*. Le plateau est affiché en ASCII sur la sortie standard.

Voici un diagramme indiquant comment sont liées les différentes classes du package CLI.

schema du couche cli de notre architecture (Fig. 10).

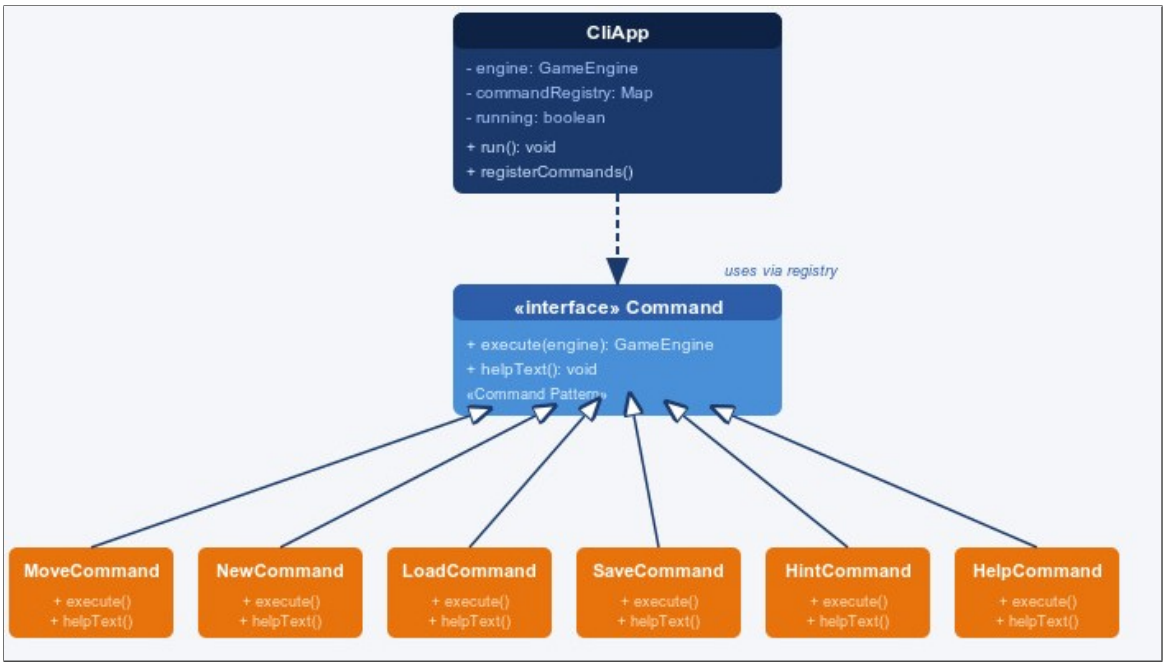


FIGURE 10 – Couche CLI

3.7 Points forts architecturaux

— **Indépendance du domaine** : notre package `core` est sans dépendance externe

— **Extensibilité de l'IA** : si l'on veut implémenter un nouvel algorithme d'IA, on étend simplement `AiPlayer`

— **Pattern Command en CLI** : on teste chaque commande indépendamment et elles sont extensibles
— **Persistance découplée** : notre `GameSaver` / `GameLoader` ne connaissent que les classes du `core`
— **Builder pour MinimaxPlayer** : on a une configuration déclarative et lisible de l'IA

3.8 Axes d'amélioration

Références

[1] Lisa Glendenning. *Mastering Quoridor*. 2006. June.

[2] M. Lloyd. *Quoridor Story : Strategy and Artificial Intelligence*. 2022.

[3] Ivona Bezakova. Board game strategies in introductory computer science, 2013. March.

[4] P. J. C. Mertens. A quoridor-playing agent, 2006. June 21.

[5] Cameron B. Browne et al. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 2012.

[6] Dipesh Koirala. Artificial intelligence for quoridor game. Master's thesis, Charles University, Faculty of Mathematics and Physics, Prague, Czech Republic, 2018.

[7] Victor Massagué Respall. Monte carlo tree search for quoridor, 2018. September.

[8] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv :1712.01815*, 2017. Fondation théorique pour l'intégration de MCTS et des réseaux de neurones.

[9] Robert C. Martin. *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall, Boston, MA, USA, 2017.

[10] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, Boston, MA, USA, 2002.

A Section en annexe

Besoins	Titre du besoin	Description du besoin	État
F1	Lancement	Lancement via CLI avec options (-h, -V, -v, -d) et gestion des flux stdout/stderr.	FAIT
F2	Configuration	Lecture et création du fichier .qoridorrc au format INI dans le HOME.	FAIT
F3	i18n	Support de l'anglais et du français via les variables LANG/LC_ALL.	FAIT
F4	Interface CLI	Démarrage auto, application des options ou chargement d'une sauvegarde.	FAIT
F5	Mode Blitz	Mode avec temps limité par joueur (-b) et réglage de la durée (-t).	FAIT
F6	Mode Contest	Lecture d'un fichier de position et réponse du meilleur coup sur stdout.	FAIT
F7	Interface Graphique	Lancement d'une fenêtre GUI via l'option -g ou –gui.	FAIT
F8	Mode IA	Remplacement d'un ou plusieurs joueurs par une intelligence artificielle.	FAIT
F9	Options Spécifiques	Paramétrage du nombre de joueurs (2-4), des murs et de la taille du plateau (3-15).	FAIT
F10	Gestion des règles	Arbitrage du jeu, validation des coups, calcul des scores et victoire.	FAIT
F11	Erreurs Utilisateur	Gestion robuste des mauvaises saisies avec messages explicites.	FAIT
F12	Undo/Redo	Historique des coups et possibilité d'annuler/rétablir pour les humains.	FAIT
F13	Conseils	Système d'aide suggérant le meilleur coup à jouer selon l'état actuel.	FAIT
F14	Gestion Blitz	Affichage du chronomètre, gestion de la pause et défaite au temps.	FAIT
F15	Shell interactif	CLI sous forme de shell avec commandes (new, save, load, show, etc.).	FAIT
F16	Sauvegarde/Quitter	Confirmation de sauvegarde lors de la fermeture si la partie est modifiée.	FAIT
F17	Édition ligne	Support de l'édition de ligne (curseur) via Readline ou équivalent.	FAIT
F18	Historique Shell	Navigation dans les anciennes commandes et recherche par le caractère "+".	FAIT
F19	Complétion	Autocomplétion des commandes via la touche Tabulation.	FAIT

TABLE 2 – Vue d'ensemble des besoins et leurs realisations

F20	Notation des coups	Représentation ASCII du plateau et syntaxe de mouvement (ex : e2-e3, e2v).	FAIT
F21	Format Sauvegarde	Import/export en format texte ASCII avec paramètres, état et historique.	FAIT
F22	Commentaires	Support des commentaires mono-ligne (#) et bloc ({ }) dans les sauvegardes.	FAIT
F23	Paramètres Jeu	Format clé-valeur pour les réglages dans le fichier de sauvegarde.	FAIT
F24	Format Plateau	Représentation visuelle simplifiée du plateau pour la restauration.	FAIT
F25	Format Historique	Liste chronologique des tours et des coups par joueur.	FAIT
F26	Module Graph	Utilisation d'une liste d'adjacence et algorithmes de graphes indépendants.	FAIT
F27	Algos Graphes	Calcul des coups valides, scores et détection de fin de partie via le graphe.	FAIT
F28	Menus GUI	Implémentation des menus 'File' et 'Game' dans l'interface graphique.	FAIT
F29	Raccourcis Clavier	Raccourcis Ctrl+N, S, O, etc. et personnalisation via configuration.	FAIT
F30	Drag'n'Drop	Manipulation des pièces et des murs à la souris dans le GUI.	FAIT
F31	IA : Temps borné	Limite de réflexion de l'IA (5s par défaut) avec coup par défaut si dépassé.	FAIT
F32	IA : Minimax	Implémentation de l'algorithme Minimax avec élagage Alpha-Beta.	FAIT
F33	IA : Fonctions Score	Au moins 3 fonctions d'évaluation différentes pour l'IA Minimax.	FAIT
F34	IA : Deepening	Mode approfondissement itératif pour optimiser la recherche.	FAIT
F35	IA : Profondeur	Option pour fixer la profondeur max de recherche de l'algorithme.	FAIT
F36	IA : MCTS	Algorithme Monte Carlo Tree Search avec fonction UCT.	FAIT
F37	IA : Machine Learning	Fonction de sélection MCTS basée sur l'apprentissage automatique (ML).	FAIT
F38	Réseau : Basique	Découverte UDP, connexion TCP, ping/pong et client/serveur simple.	FAIT
F39	Réseau : Multi-clients	Serveur gérant plusieurs parties, scoreboard et validation anti-triche.	FAIT
F40	Réseau : Invitations	Système avancé d'invitations (accept/decline) et gestion des statuts joueurs.	Partiellement FAIT

TABLE 3 – Vue d'ensemble des besoins et leurs réalisations